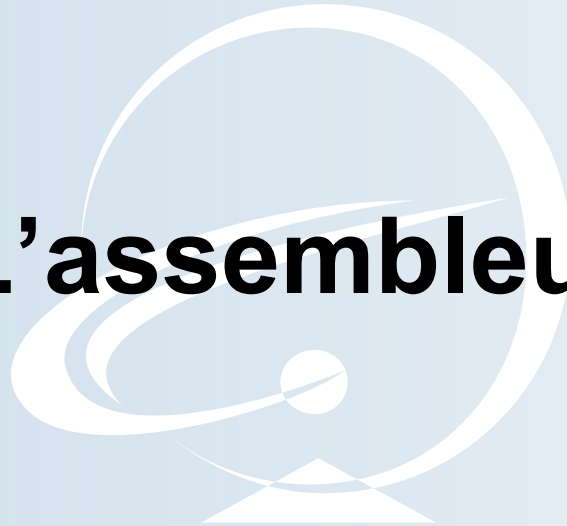




L'assembleur – niveau 1



```
.model small
.stack 100h
.data
    bonjour    db    "Hello world!$"
.code
main    proc
    mov     AX, @data
    mov     DS, AX
    mov     DX, offset bonjour
    mov     AX, 0900h
    int     21h
    mov     AX, 4C00h
    mov     21h
main    endp
end main
```



Objectifs de ce module

En suivant ce module vous allez :



- **Apprendre** l'organisation interne d'un 8086, le CPU qui servait de base au premier PC d'IBM.
- **Comprendre** l'utilité des différents registres du CPU.
- **Découvrir** les bases du langage Assembleur du 8086 au travers de l'émulateur EMU 8086.





Plan du module

Voici les parties que nous allons aborder :



- Présentation du 8086.
- Présentation de l'environnement Emu8086.
- Manipulation de base.
- Les sauts.





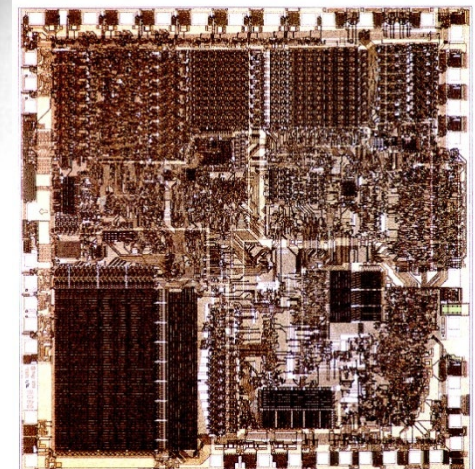
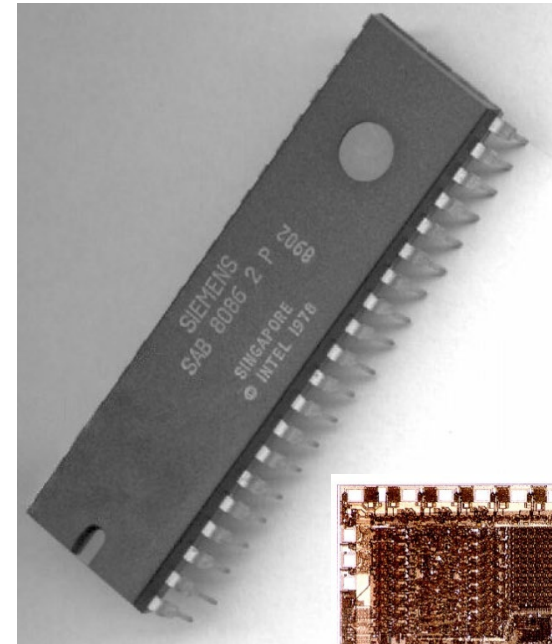
Présentation du 8086



Plan de la partie

Voici les chapitres que nous allons aborder :

- Vue externe du 8086.
- Vue interne du 8086.
- Les registres généraux.
- Les registres d'index.
- Les registres de segment.
- Les registres pointeurs.
- Le registre IP.
- Le registre d'état.





Vue externe du 8086

Le packaging

- Le 8086 se présente sous la forme d'un boîtier DIP.





Vue externe du 8086

Les différentes broches

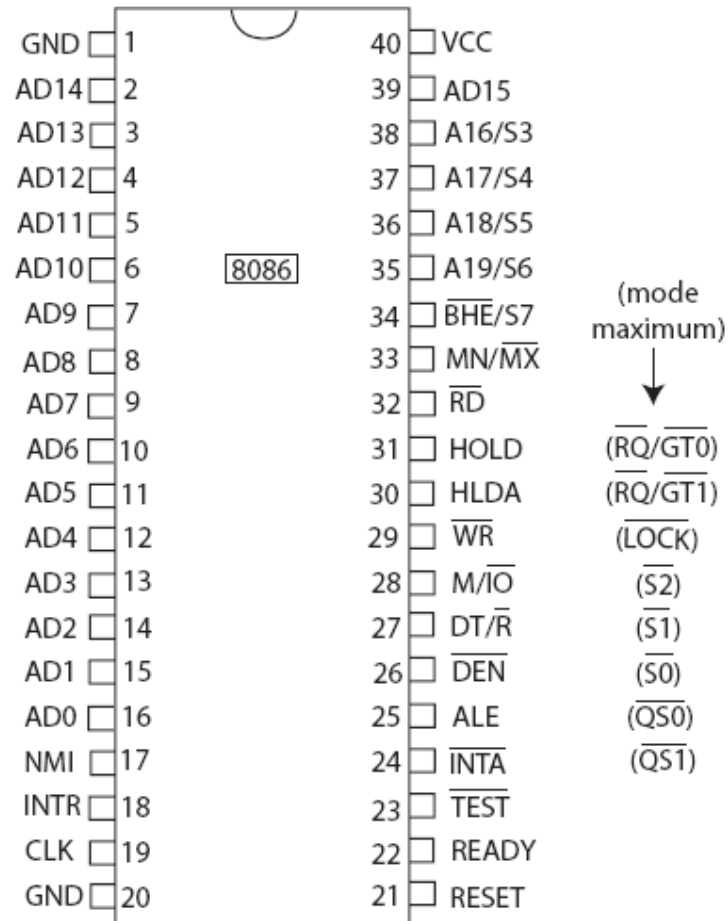
- Ce processeur est doté d'un bus d'adresse et d'un bus de données qui sont **multiplexés** (certaines pattes transmettent à certains moments un bit d'adresse et à d'autres moments un bit de donnée) :
 - Nous avons 16 broches qui permettent de transporter les données (AD0 ... AD15), la largeur du bus de données est de 16 bits.
 - Nous avons 20 broches qui permettent de véhiculer les adresses (AD0 ... A19), le 8086 peut donc adresser 2^{20} (=1048576) positions mémoire différentes contenant chacune 1 octet (8 bits) : la mémoire a donc au plus une capacité de 1 Mo.





Vue externe du 8086

Les différentes broches





Vue externe du 8086

Les différentes broches

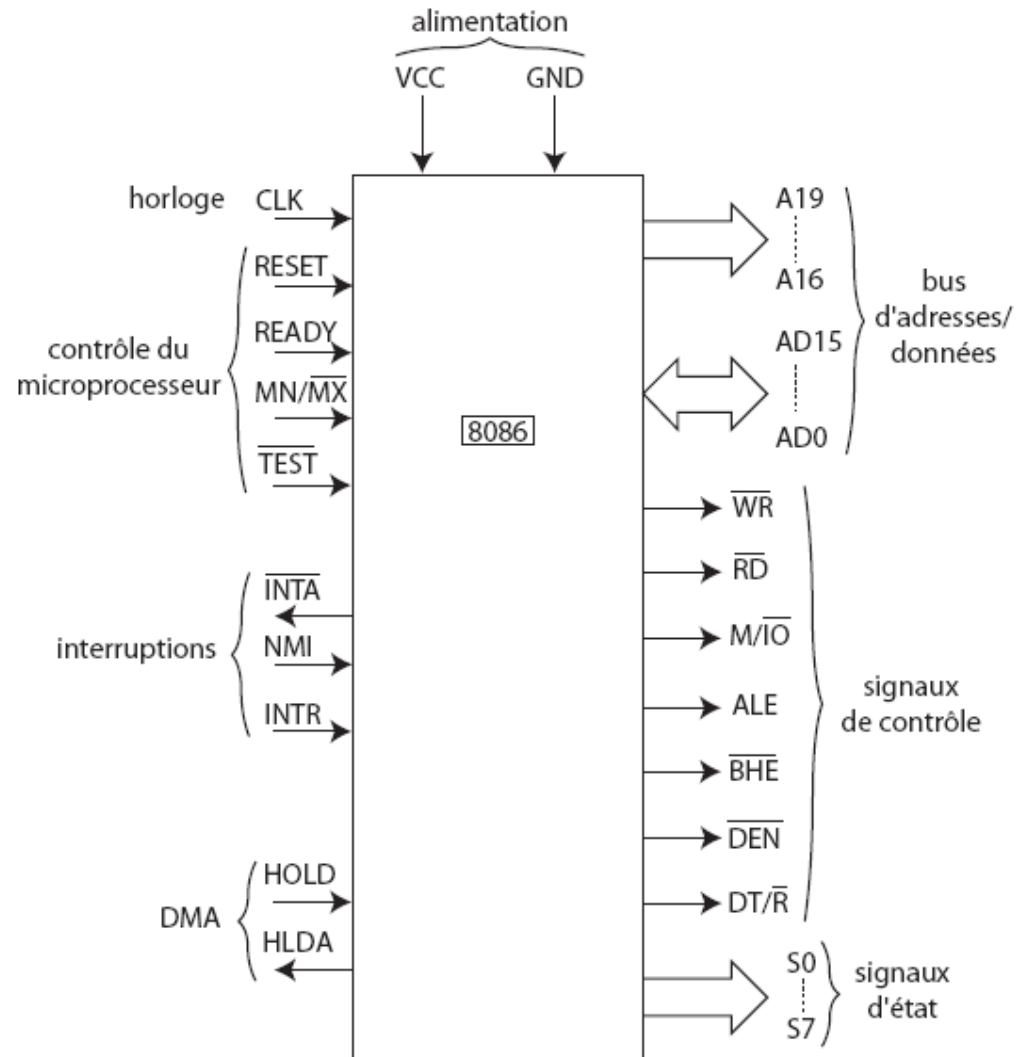
- Le 8086 peut également être représenté d'une manière fonctionnelle.
- Cela permet de mettre en évidence les différents signaux en entrée et en sortie du processeur.





Vue externe du 8086

Les différentes broches





Vue interne du 8086

Le schéma bloc

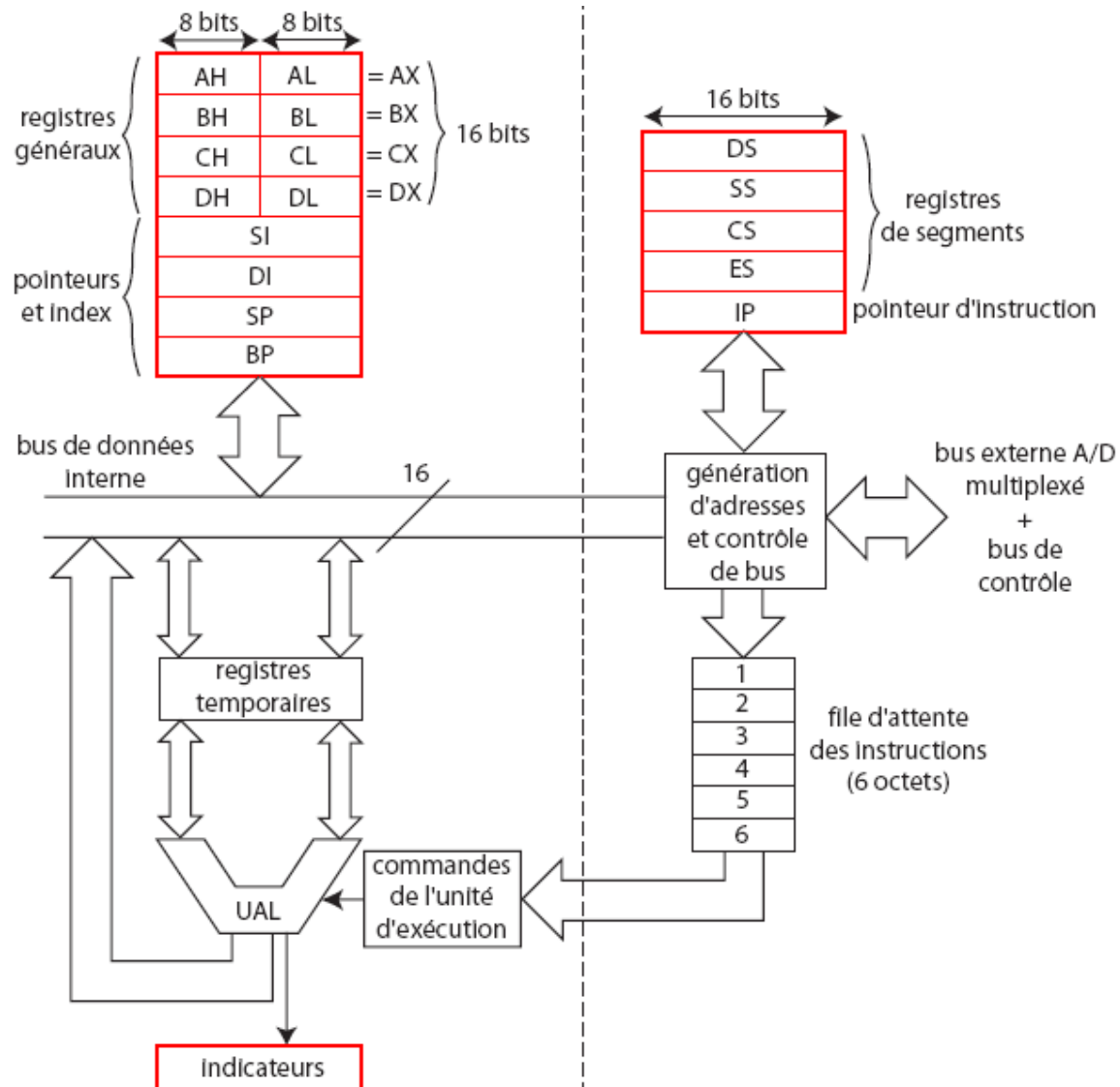
- Dans les *databooks* qui décrivent les processeurs, ces derniers sont souvent représentés sous forme de schémas bloc afin de mettre en évidence :
 - les gros ensembles de composants ;
 - les bus qui relient ces gros ensembles.





Vue interne du 8086

Le schéma bloc





Vue interne du 8086

Les registres et le bus interne de données

- L'analyse du schéma bloc montre que le 8086 est organisé autour d'un bus interne de données d'une largeur de 16 bits.
- Autour de ce bus, nous trouvons :
 - des registres de 16 bits permettant de stocker des données ;
 - des registres de 16 bits permettant de stocker une part d'une adresse (qui est codée sur 20 bits) ;
 - des registres temporaires qui débouchent sur une ALU ;
 - un bloc qui permet de former une adresse sur 20 bits à partir de morceaux d'adresse codés sur 16 bits.





Les registres généraux

Les deux manières de voir les registres généraux

- Le 8086 dispose de registres généraux qui peuvent être vus de deux manières :
 - Si nous manipulons des nombres codés sur 16 bits (ce que nous désignons sous le terme de **mot**), nous utilisons les registres en mode 16 bits et nous disposons alors de 4 registres appelés
AX, BX, CX et DX.
 - Si nous manipulons des nombres codés sur 8 bits (ce que nous désignons sous le terme d'**octet**), nous utilisons les registres en mode 8 bits et nous disposons de 8 registres appelés

AH, AL, BH, BL, CH, CL, DH et DL.





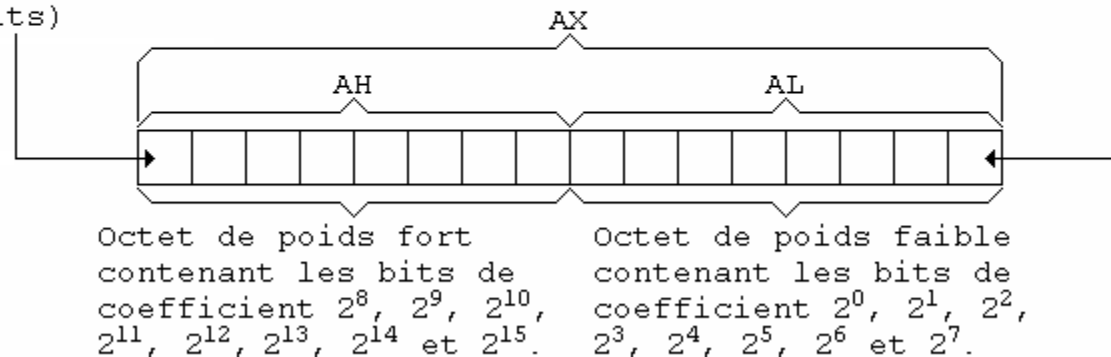
Les registres généraux

Les deux manières de voir les registres généraux

- H est mis pour "High" et L pour "Low" afin de rappeler que AH contient l'octet de poids fort du mot de 16 bits contenu dans AX et que AL contient l'octet de poids faible de ce même mot de 16 bits.

Bit ayant un coefficient de 2^{15} , il est donc qualifié de bit de poids fort (pour un mot de 16 bits)

Bit ayant un coefficient de 2^0 , il est donc qualifié de bit de poids faible





Les registres généraux

Les deux modes d'utilisation

- Ces registres sont qualifiés de généraux car on peut les utiliser de manière indifférenciée pour les calculs arithmétiques et logiques mais ils ont un rôle bien précis dans d'autres circonstances comme nous le verrons un peu plus loin :
 - **AX**, appelé l'**accumulateur**, est utilisé pour stocker les résultats de certains calculs arithmétiques et logiques ;
 - **BX**, appelé la **base**, est utilisé comme registre de base pour des données dans le segment pointé par le registre de segment ES ;





Les registres généraux

Les deux modes d'utilisation

- Ces registres sont qualifiés de généraux car on peut les utiliser de manière indifférenciée pour les calculs arithmétiques et logiques mais ils ont un rôle bien précis dans d'autres circonstances comme nous le verrons un peu plus loin :
 - **CX**, appelé le **compteur**, est utilisé pour les boucles ;
 - **DX**, appelé le **registre de données**, contient l'adresse des ports d'entrée/sortie (pour les instructions IN et OUT) et sert également d'extension à AX lorsqu'on manipule des données sur 32 bits.





Les registres d'index

La copie de zone de mémoire

- Le 8086 possède deux registres d'index, notés SI et DI, qui sont utilisés pour indexer les éléments d'un tableau. Dans les instructions de mouvement de chaînes d'octets, ils sont utilisés simultanément :
 - **SI** (*S*ource *I*ndex) indexe les caractères de la chaîne émettrice ;
 - **DI** (*D*estination *I*ndex) indexe les caractères de la chaîne émettrice.





Les registres de segment

Les 4 segments

- Le 8086 peut adresser une mémoire de 1 Mo grâce à son bus d'adresse de 20 bits de large.
- Cette mémoire est découpée en tranches de 64 Ko. maximum, appelées **segments** et référencées par 4 registres.
- Remarque : La notion de segmentation de la mémoire sera présentée de façon plus approfondie, l'an prochain, dans le cours de théorie des système d'exploitation.





Les registres de segment

Les 4 segments

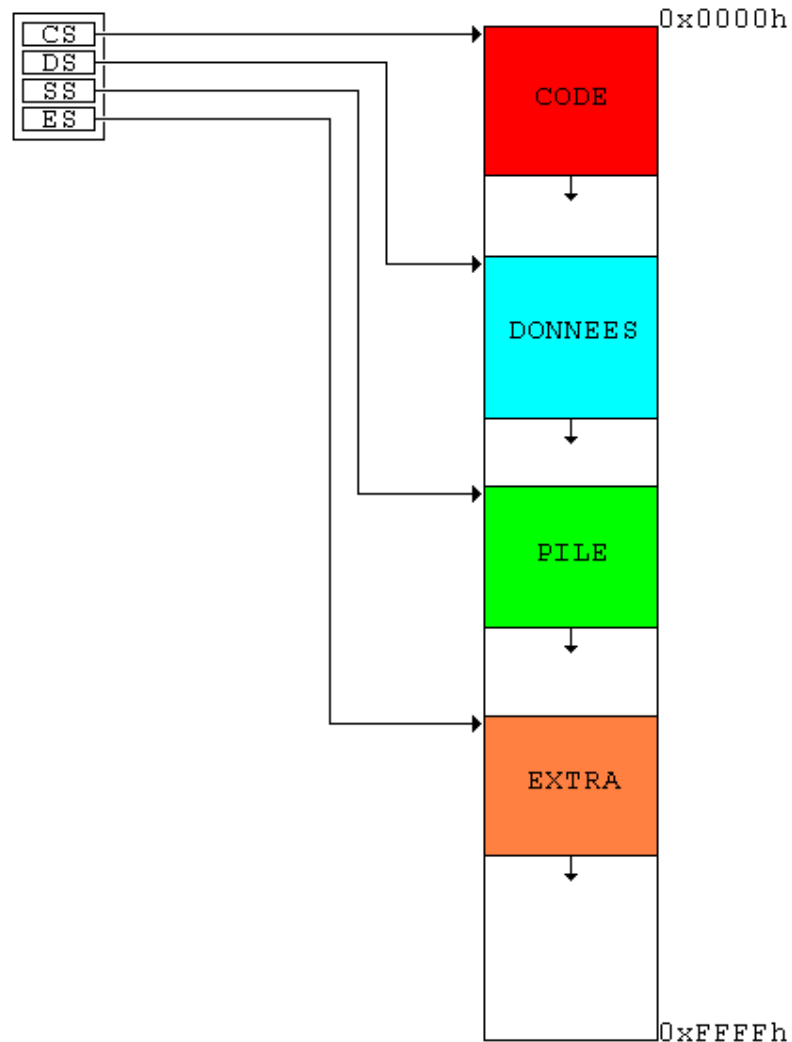
- Les 4 registres segment sont les suivants :
 - **CS** (*C*ode *S*egment) pointe sur la base du segment qui contient le code (les instructions que doit exécuter le processeur) ;
 - **DS** (*D*ata *S*egment) pointe sur la base du segment contenant les données (variables, tableaux ...) ;
 - **SS** (*S*tack *S*egment) pointe sur la base du segment qui contient la pile gérée par les registres SP et BP ;
 - **ES** (*E*xtra *S*egment) pointe sur la base d'un segment supplémentaire qui est généralement utilisé pour compléter le segment de données.





Les registres de segment

Les 4 segments





Les registres pointeurs

Le sommet et la base de la pile

- Le 8086 possède deux registres pointeurs, notés SP et BP, qui sont utilisés pour gérer la pile :
 - **SP** (*Stack Pointer*) pointe sur le sommet de la pile et il est mis à jour automatiquement par les instructions d'empilement et de dépilement ;
 - **BP** (*Base Pointer*) pointe la base de la région de la pile contenant les données accessibles (variables locales, paramètres,...) à l'intérieur d'une procédure. Il doit être mis à jour par le programmeur.





Les registres pointeurs

La gestion de la pile

- La pile est une zone de mémoire qui permet de conserver de manière temporaire des données (par exemple, l'état des registres lors d'un appel de procédure).

- Cette pile est de type LIFO (*Last In First Out*) et elle est manipulé par des instructions telles que :
 - PUSH pour empiler une valeur de 16 bits (SP est décrémenté automatiquement de 2) ;
 - POP pour dépiler une valeur de 16 bits (SP est incrémenté automatiquement de 2).

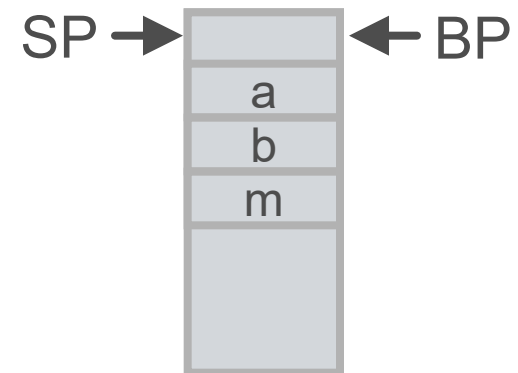




Les registres pointeurs

La gestion de la pile

- La pile permet également de conserver les arguments d'une procédure. On utilise alors le registre BP (géré par le programmeur) pour accéder à ces arguments (cela permet de se déplacer dans la pile pour accéder aux informations sans avoir à modifier SP).
- Si on considère la procédure `min (int a, int b, int* m)`, on a donc la représentation suivante en mémoire :
 - a est en $SP + 2$
 - b est en $SP + 4$
 - m est en $SP + 6$





Le registre IP

Le pointeur d'instructions ou le compteur ordinal

- Le registre **IP** (*Instruction Pointeur*) est appelé « pointeur d'instruction » ou « compteur ordinal ».
- Comme son nom l'indique, il permet de pointer une case mémoire dans le segment de code afin que le 8086 puisse charger la prochaine instruction à exécuter.
- Déjà évoqué dans le module 5, diapositive 49.





Le registre indicateur

Les différents bits d'état du processeur et de l'ALU

- Le registre indicateur contient des drapeaux (*flags*) qui sont mis à 0 ou 1 par le processeur (à la suite de l'exécution d'une instruction) ou par le programmeur pour modifier le mode de fonctionnement du processeur :
 - Le bit **Retenue** (*Carry*), encore appelé **Dépassement**, est mis à 1 par l'ALU lorsque, par exemple, une addition de deux nombres de 16 bits (respectivement de 8 bits) produit un résultat sur 17 bits (respectivement sur 9 bits). Il peut être utilisé pour effectuer des additions sur 32 bits (voir diapositive 31 et module 7).
 - Le bit **Parité** (*Parity*) est mis à 1 par l'ALU lorsque le résultat d'une opération contient un nombre pair de 1.





Le registre indicateur

Les différents bits d'état du processeur et de l'ALU

- Le registre indicateur contient des drapeaux (*flags*) qui sont mis à 0 ou 1 par le processeur (à la suite de l'exécution d'une instruction) ou par le programmeur pour modifier le mode de fonctionnement du processeur :
 - Le bit **Retenue Intermédiaire** (*Auxiliary carry*) est mis à 1 par l'ALU lorsqu'il y a une retenue du quartet de poids faible vers le quartet de poids fort ou dans le quartet de poids fort (les quartets concernent uniquement l'octet de poids faible)
 - Le bit **Zéro** (*Zero*) est mis à 1 par l'ALU quand le résultat d'une opération est 0.





Le registre indicateur

Les différents bits d'état du processeur et de l'ALU

- Le registre indicateur contient des drapeaux (*flags*) qui sont mis à 0 ou 1 par le processeur (à la suite de l'exécution d'une instruction) ou par le programmeur pour modifier le mode de fonctionnement du processeur :
 - Le bit **Signe** (*Signe*) est mis à 1 par l'ALU lorsque le bit de poids fort du résultat d'une opération est égal à 1.
 - Le bit **Débordement** (*Overflow*) est mis à 1 par l'ALU lorsqu'un débordement de la capacité de stockage du résultat est survenu et qu'un chiffre significatif a été perdu.





Le registre indicateur

Les différents bits d'état du processeur et de l'ALU

- Le registre indicateur contient des drapeaux (*flags*) qui sont mis à 0 ou 1 par le processeur (à la suite de l'exécution d'une instruction) ou par le programmeur pour modifier le mode de fonctionnement du processeur :
 - Le bit **Pas-à-pas** (*Trap*) est mis à 1 par le programmeur pour forcer le processeur à fonctionner en mode pas-à-pas (pour mettre au point des cartes électroniques par exemple).
 - Le bit **Interruptions externes** (*Interrupt Enable*) est mis à 1 par le programmeur pour spécifier au processeur qu'il doit tenir compte du signal d'interruption envoyé par les autres circuits électroniques de la carte mère.





Le registre indicateur

Les différents bits d'état du processeur et de l'ALU

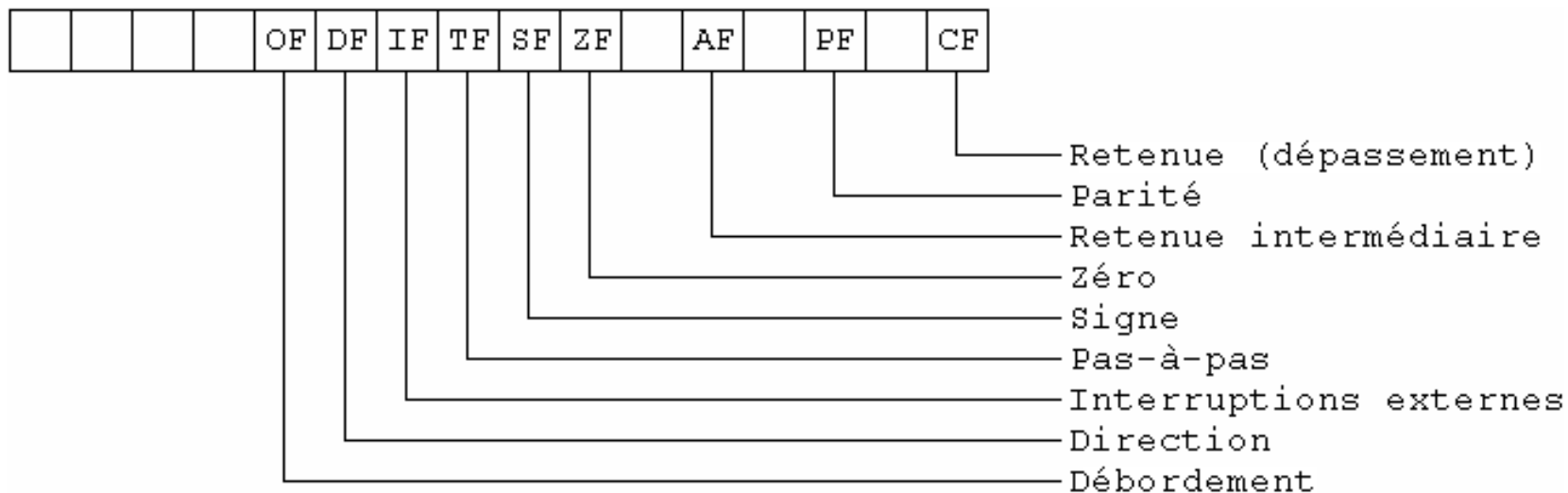
- Le registre indicateur contient des drapeaux (*flags*) qui sont mis à 0 ou 1 par le processeur (à la suite de l'exécution d'une instruction) ou par le programmeur pour modifier le mode de fonctionnement du processeur :
 - Le bit **Direction** (*Direction*) est mis à 1 par le programmeur pour indiquer que le traitement des blocs de données s'effectuera par ordre décroissant des adresses (par exemple, la recopie d'une chaîne de caractères).





Le registre indicateur

Schéma de synthèse





Le registre indicateur

Remarque concernant le bit CF

- On peut utiliser le bit CF (retenu, dépassement) afin de l'utiliser pour effectuer des additions sur 32 bits
- Si on considère deux nombres A et B codés sur 32 bits, on va additionner ces deux nombres en procédant de la manière suivante :
 - Addition des 16 bits de poids faible de A et de B ;
 - Addition des 16 bits de poids fort de A et de B avec le bit CF.
- Ce bit CF permet donc de propager la retenue entre les bits n°15 et le bits n°16 de l'addition.





Pause-réflexion sur cette 1^{ère} partie

Avez-vous des questions ?





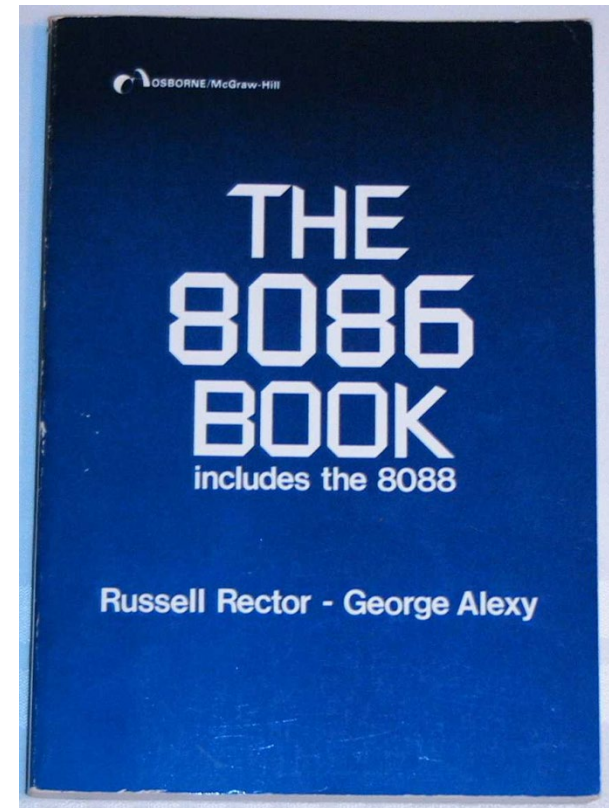
Présentation de l'environnement Emu8086



Plan de la partie

Voici les chapitres que nous allons aborder :

- L'environnement de programmation.
- L'environnement d'exécution.
- L'assemblage et la notion d'opcode.
- L'exécution.

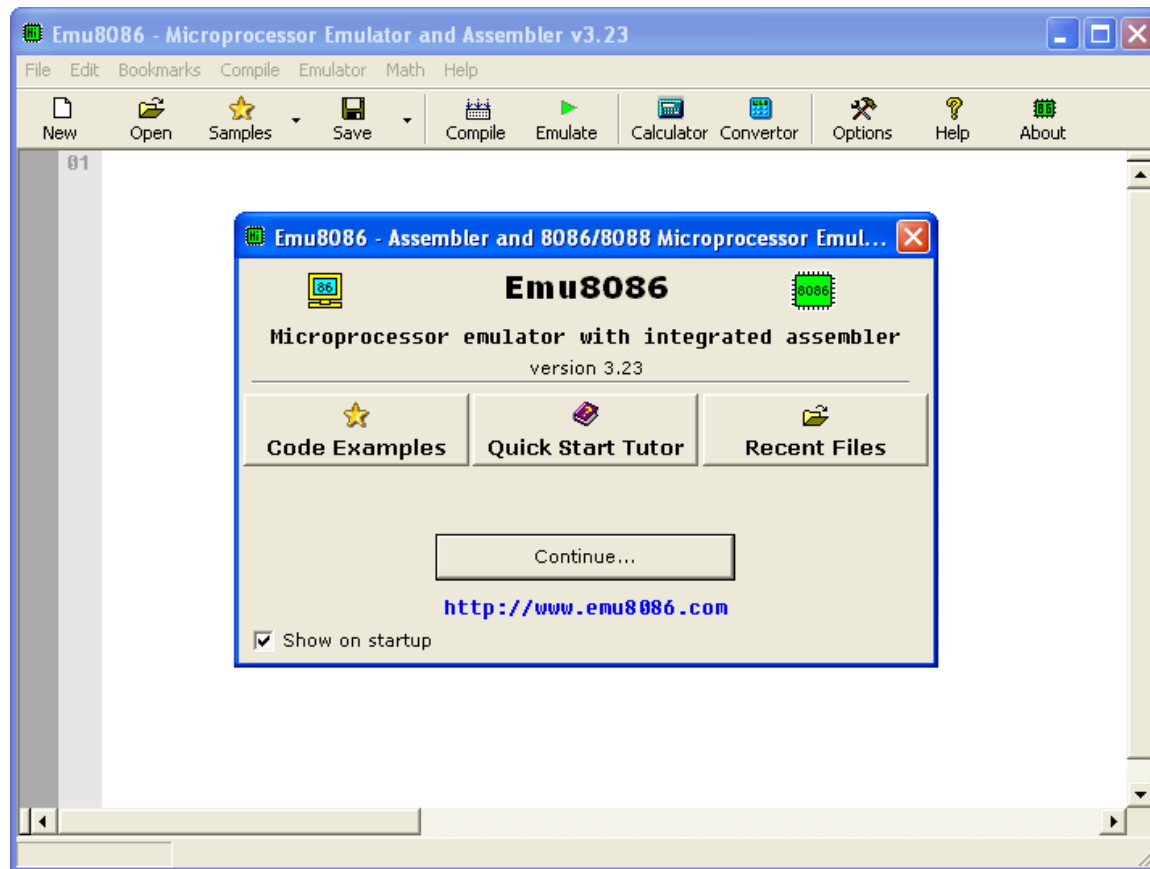




L'environnement de programmation

Le démarrage

- Lorsqu'on démarre Emu 8086, l'application propose une interface comparable à celle de la figure ci-dessous





L'environnement de programmation

Le premier programme assembleur

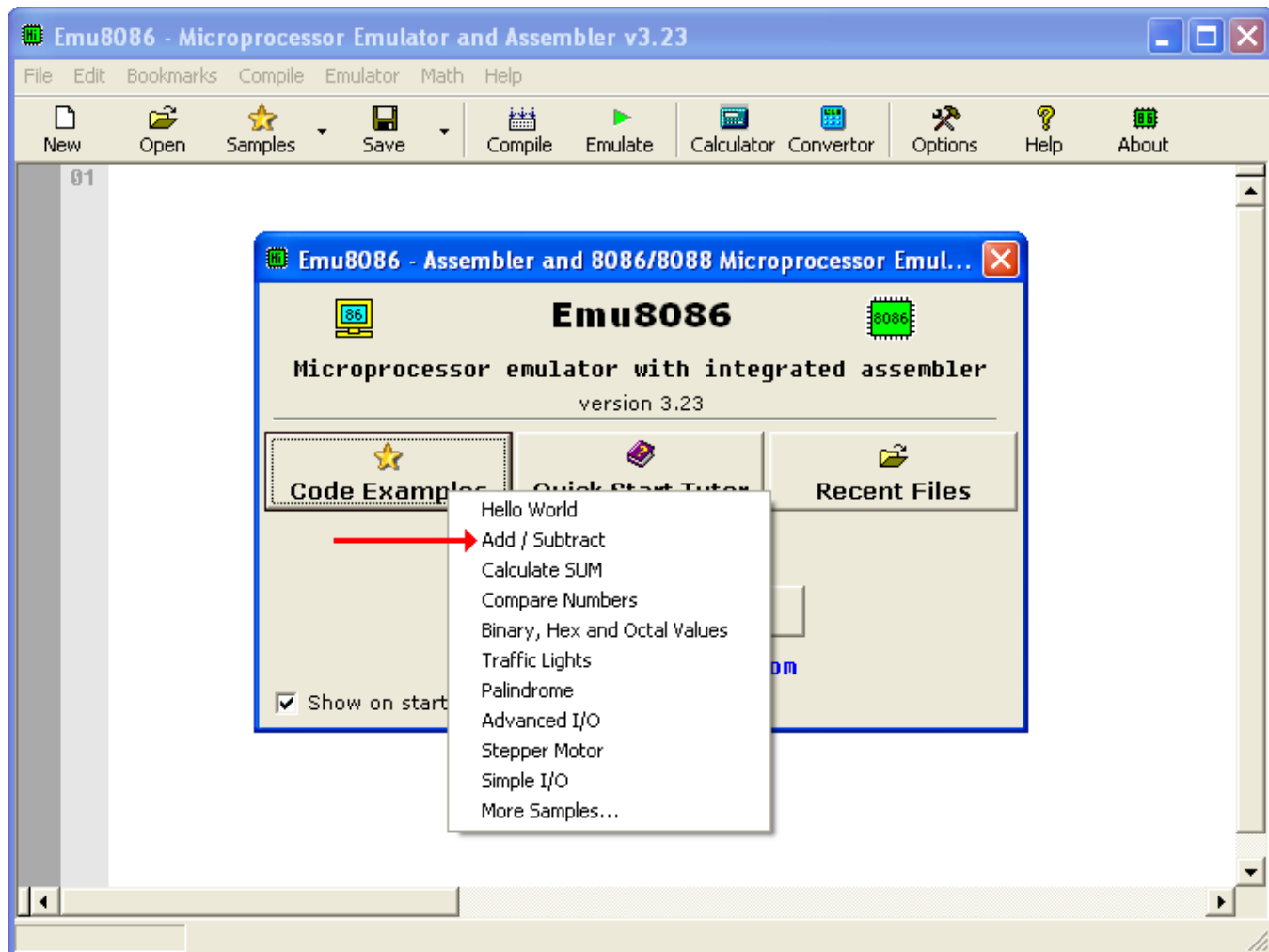
- Si nous appuyons sur le bouton "Code Examples", nous avons accès à une liste d'exemple.
- Le programme Hello World étant trop complexe à ce stade (il faut d'abord décrire le fonctionnement de la carte vidéo pour comprendre cet exemple), nous allons nous orienter vers le programme "Add / Subtract"





L'environnement de programmation

Le premier programme assembleur





L'environnement de programmation

Le premier programme assembleur

- Le code du programme apparaît alors dans la fenêtre principale.

```
01 org 100h
02
03 MOV AX, 5
04 MOV BX, 10
05
06 ; 5 + 10 = 15 (000Fh)
07 ADD AX, BX
08
09 ; 15 - 1 = 14 (000Eh)
10 SUB AX, 1
11
12 ret
13
```





L'environnement de programmation

Description du premier code

org 100h

MOV AX, 5

MOV BX, 10

; 5 + 10 = 15 (000Fh)

ADD AX, BX

; 15 - 1 = 14 (000Eh)

SUB AX, 1

ret

Cette instruction est affichée en noir car il s'agit d'une directive d'assemblage (et non d'une instruction pour le processeur) utilisée pour créer un exécutable (un .com) qui tient dans un seul segment (à voir plus tard).

On met la valeur décimale 5 dans le registre AX du processeur

On met la valeur décimale 10 dans le registre BX du processeur

On demande au processeur d'ajouter le contenu du registre BX au contenu du registre AX : le résultat de cette somme (15 soit 000F en hexa décimal) se trouve donc dans le registre AX.

On demande au processeur de soustraire 1 au contenu du registre AX : l'issue de cette instruction, le registre AX contient 14 (soit 000E en hexadécimal).

En assembleur, les lignes de commentaires commencent généralement par un point-vigule

ret est une instruction qui provoque le branchement vers d'autres instructions propres au système d'exploitation afin de mettre fin à l'exécution du programme assembleur et de rendre la main à l'OS.

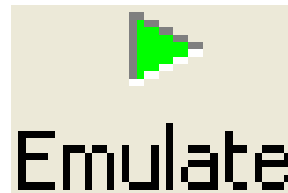




L'environnement d'exécution

L'exécution du premier programme

- Nous pouvons appuyer sur le bouton "Emulate" afin de compiler le programme (créer un exécutable - un .com - qui se trouve dans le répertoire MyBuild, sous le répertoire d'installation d'Emu 8086) et de lancer le module de simulation.



- La fenêtre de gauche apparaît, nous pouvons appuyer sur les boutons du bas afin de faire apparaître d'autres fenêtres qui nous aiderons à comprendre le fonctionnement de ce programme au niveau du pseudo-processeur.





L'exécution du premier programme





L'environnement d'exécution

L'adressage du code objet en mémoire

- La partie gauche de la fenêtre principale comporte les différents registres (AX, BX, IP ...) et une représentation du programme assemblé en mémoire vive.

- Nous constatons que sous le bouton "memory (1K) at", nous avons deux nombres hexadécimaux :
 - le premier est le nombre stocké dans le registre CS
 - le second à celui stocké dans IP.





L'environnement d'exécution

L'adressage du code objet en mémoire

- Pour désigner une instruction contenue dans une case de la mémoire de 1 Mo., nous avons besoin d'une adresse codée sur 20 bits (cela dépasse les capacités des registres internes du 8086).
- Pour contourner ce problème, l'adresse est codée avec deux registres :
 - le registre CS contient l'adresse de la base du segment de code
 - le registre IP contient une adresse relative à cette base (on parle de déplacement ou d'offset).
- L'adresse absolue est obtenue en faisant **$CS * 16 + IP$** .





L'environnement d'exécution

L'adressage du code objet en mémoire

The screenshot displays the Emu8086 environment. On the left, the 'Registers' panel shows the state of various registers. The 'memory (1K) at:' panel shows the current memory address and its contents. A calculation on the right shows the address calculation for the first instruction.

Registers:

	H	L
AX	00	00
BX	00	00
CX	00	0C
DX	00	00
CS	0B56	
IP	0100	
SS	0B56	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0B56	
ES	0B56	

memory (1K) at:

Offset:	Hex	Dec	ASCII
0100:	B8	184	
0101:	05	005	
0102:	00	000	
0103:	BB	187	>>
0104:	0A	010	
0105:	00	000	
0106:	03	003	
0107:	C3	195	Ã
0108:	2D	045	-
0109:	01	001	
010A:	00	000	
010B:	C3	195	Ã
010C:	00	000	
010D:	00	000	
010E:	00	000	

Calculation:

$$0B560 + 0100 = 0B660$$

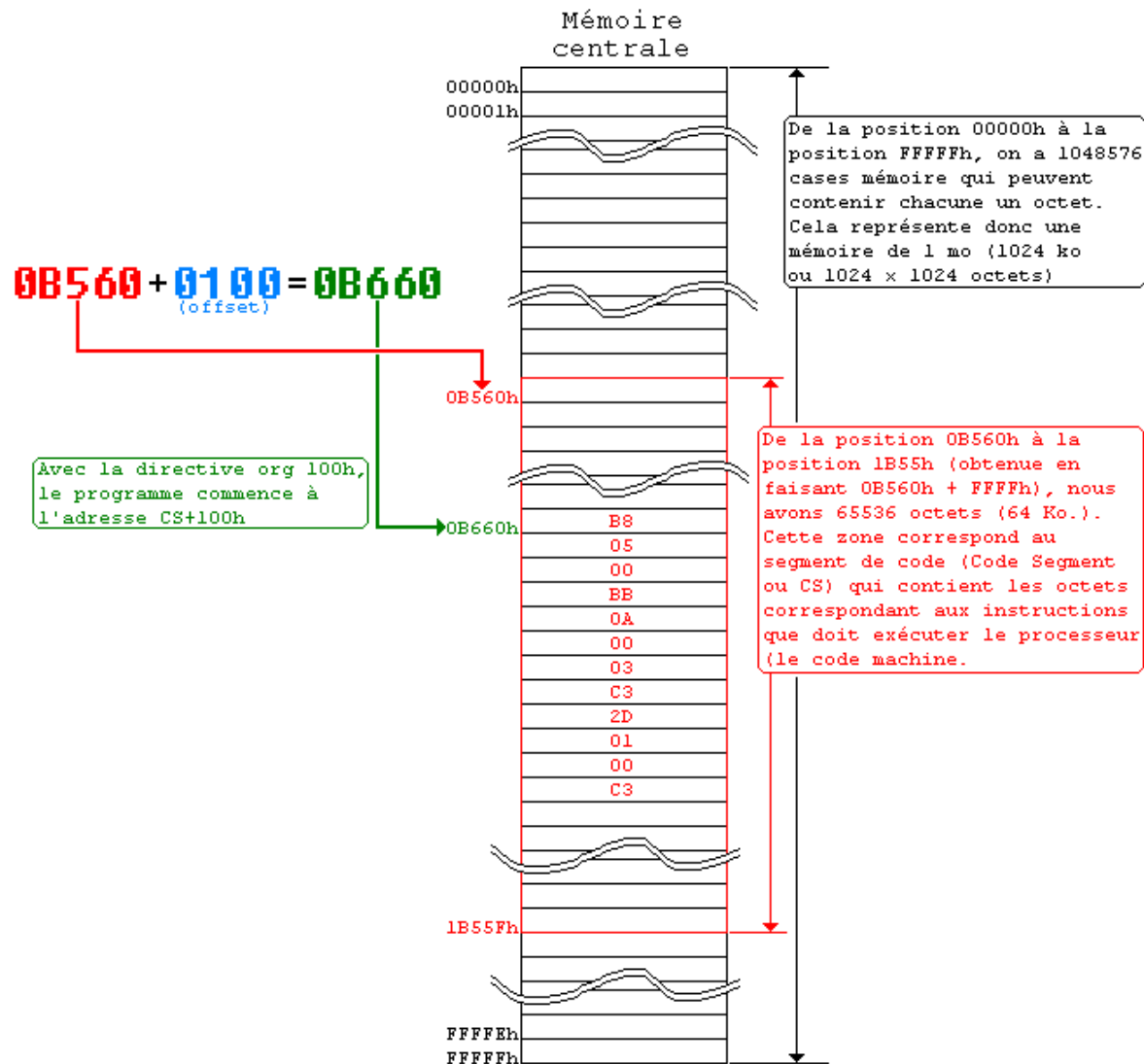
The calculation shows the base address 0B560 (with a multiplier of 10h) plus the offset 0100, resulting in the final address 0B660.

Le registre IP (Instruction Pointer) pointe sur la première instruction du code machine qui se trouve à CS+100h.



L'environnement d'exécution

L'adressage du code objet en mémoire





L'assemblage et la notion d'opcode

La notion de mnémonique et d'opcode

- Chaque instruction assembleur (mnémonique) possède une correspondance en langage machine, appelée **code opérateur** ou **opcode**, qui peut être codé sur un ou plusieurs octets selon le besoin.
- Cette correspondance est spécifiée par le constructeur - Intel - dans son *databook* ce qui permet au programmeur de développer leur assembleur et/ou leur émulateur.





L'assemblage et la notion d'opcode

Instruction a à dressage immédiat et à registre

- Les instructions **opération registre, valeur** sont dites **à adressage immédiat** car on indique "en dur" la valeur qui doit être chargée dans le registre
- Les instructions **opération registre, registre** sont dites **de registres** car les données, sur lesquelles les opérations sont effectuées, sont contenues dans les registres.
- Dans ces deux cas de figures, il n'y a pas de construction d'adresses sur 20 bits puisqu'on ne fait pas intervenir la mémoire, ces instructions sont donc rapides à exécuter.





L'assemblage et la notion d'opcode

Extrait du databook de l'Intel 8086

D'après le databook (écrit par Intel) qui décrit le fonctionnement du processeur 8086, nous avons les correspondances suivantes entre les mnémoniques et les octets (ou les groupes d'octets) :

- * Un mov immédiat vers un registre se traduit par un bloc de deux octets si w=0 ou de trois octets si w=1

`1011wreg` `data` `data si w=1`

avec reg=000 pour AL (si w=0) ou AX (si w=1)
et reg=011 pour BL (si w=0) ou BX (si w=1)

- * Un add entre deux registres se traduit par un bloc de deux octets

`000000dw` `modreg r/m`

avec d=1 (si c'est vers la destination) ou d=0 (si c'est depuis une source). w=0 (addition sur 8 bits) ou w=1 (addition sur 16 bits). mod=11 car l'addition fait intervenir deux registres : reg est égal au code du premier registre et r/m au code du deuxième registre

- * Un sub immédiat depuis l'accumulateur (le registre AX) se traduit par un bloc de deux octets si w=0 ou de trois octets si w=1

`0010110w` `data` `data si w=1`

- * Un ret (un retour d'appel de programme, de fonction ou de procédure) se traduit par un octet si le saut s'effectue dans le segment CS

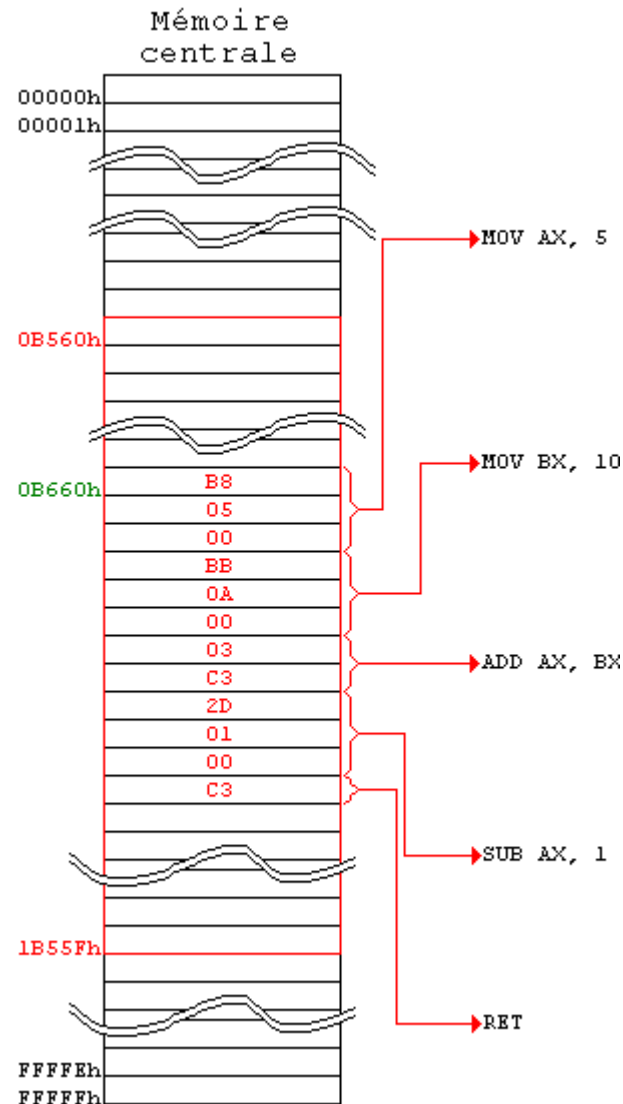
`11000011`





L'assemblage et la notion d'opcode

Correspondance opcode - mnémonique





L'assemblage et la notion d'opcode

Description du code

Cette instruction signifie qu'on transfère de façon immédiate le nombre 5 vers le registre AX (l'accumulateur). Comme AX est un registre sur 16 bits, cela signifie qu'on manipule un mot donc $w=1$ et cette instruction occupera trois octets d'après le databook. En outre, comme il s'agit du registre AX, nous avons $reg=000$ (cf. databook). Enfin, comme les processeurs Intel sont de type "bigendian", l'octet de poids fort est placé après l'octet de poids faible : le nombre 5, qui aurait dû être codé par 0005h, est codé par 0500h.

Cette instruction se traduit donc par 10111000 00000101 00000000 soit B8 05 00

Cette instruction signifie qu'on transfère de façon immédiate le nombre 10 vers le registre BX. Comme BX est un registre sur 16 bits, nous manipulons un mot donc $w=1$ et l'instruction occupera 3 octets. Comme nous utilisons le registre BX, nous avons $reg=011$. Comme le 8086 est un processeur "bigendian", 10 se code sous la forme 0A00h au lieu de 000Ah.

Cette instruction se traduit donc par 10111011 00001010 00000000 soit BB 0A 00

Cette instruction signifie qu'on fait une addition sur 16 bits d'un registre BX vers un autre registre AX, nous avons donc $w=1$ et $d=1$. Comme il s'agit d'une addition entre deux registres, nous avons le champ mod égal à 11 et le champ r/m (qui correspond à la destination) est égal au code du registre AX soit 000. Enfin le champ reg est égal au code du registre BX soit 011.

Cette instruction se traduit donc par 00000011 11000011 soit 03 C3

Cette instruction signifie qu'on fait une soustraction immédiate sur 16 bits depuis l'accumulateur (le registre AX), nous avons donc $w=1$ et le 1^{er} octet de l'instruction qui est codé sur 3 octets. Comme le 8086 est un processeur "bigendian", 1 se traduit par 0100h au lieu de 0001h.

Cette instruction se traduit donc par 00101101 00000001 00000000 soit 2D 01 00

Cette instruction signifie qu'on fait un retour vers le programme qui a appelé ce code machine (dans notre cas de figure, il s'agit de l'OS). Concrètement cela se traduit par un saut dans la pile de code CS (on reste dans la même pile) vers l'instruction qui rend la main à l'OS. Cette instruction se code donc simplement sur octet.

Cette instruction se traduit donc par 11000011 soit C3





Exécution

Exécution de la première instruction

- Nous pouvons alors appuyer sur le bouton "Single Step" afin de lancer une exécution pas-à-pas du programme.
- La première instruction (MOV AX, 5) est chargée, décodée et exécutée par le processeur
- Les changements sont les suivants :
 - Le registre AX contient la valeur 5 ;
 - Le registre IP contient la valeur 103h (IP passe de 100h à 103h car MOV AX, 5 est codé sur 3 octets) ;
 - La prochaine instruction pointée par IP (MOV BX, 10) est surlignée dans les différents écrans.





Exécution

Exécution de la première instruction

Emulator: noname.com - 8086 Microprocessor Emulator

File Math Debug View Virtual Devices Virtual Drive Help

Load Reload Single Step Run step delay ms: 0

Registers

	H	L
AX	00	05
BX	00	00
CX	00	0C
DX	00	00
CS	0B56	
IP	0103	
SS	0B56	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0B56	
ES	0B56	

memory (1K) at: 0B56 : 0100 Disassemble: 0B56 : 0100

Offset: Hex Dec ASCII

Offset	Hex	Dec	ASCII
0100	B8	184	
0101	05	005	
0102	00	000	
0103	B8	187	
0104	0A	010	
0105	00	000	
0106	03	003	
0107	C3	195	Ã
0108	2D	045	-
0109	01	001	
010A	00	000	
010B	C3	195	Ã
010C	00	000	
010D	00	000	
010E	00	000	

MOV AX, 00005h
MOV BX, 0000Ah
ADD AX, BX
SUB AX, 00001h
RET
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL

User Screen Actual Source ALU Stack FLAGS

Stack

Address	Value
0B56:FFFE	0000
0B56:FFFC	0000
0B56:FFFA	0000
0B56:FFF8	0000
0B56:FFF6	0000
0B56:FFF4	0000
0B56:FFF2	0000
0B56:FFF0	0000
0B56:FFEE	0000
0B56:FFEC	0000
0B56:FFEA	0000
0B56:FFE8	0000
0B56:FFE6	0000
0B56:FFE4	0000
0B56:FFE2	0000
0B56:FFE0	0000
0B56:FFDE	0000
0B56:FFDC	0000

Actual Emulated Source...

```
01 org 100h
02
03 MOV AX, 5
04 MOV BX, 10
05
06 ; 5 + 10 = 15 (000Fh)
07 ADD AX, BX
08
09 ; 15 - 1 = 14 (000Eh)
10 SUB AX, 1
11
12 ret
13
14
15
```

ALU - Arithmetic & Logical Unit

F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0





Exécution

Exécution de la première instruction

Emulator: noname.com - 8086 Microprocessor Emulator

File Math Debug View Virtual Devices Virtual Drive Help

Load Reload Single Step Run step delay ms: 0

Registers

	H	L
AX	00	05
BX	00	00
CX	00	0C
DX	00	00
CS	0B56	
IP	0103	
SS	0B56	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0B56	
ES	0B56	

memory (1K) at: 0B56 : 0100

Disassemble: 0B56 : 0100

Offset	Hex	Dec	ASCII
0100	B8	184	.
0101	05	005	
0102	00	000	
0103	B8	187	>
0104	0A	010	
0105	00	000	
0106	03	003	
0107	C3	195	Ã
0108	2D	045	-
0109	01	001	
010A	00	000	
010B	C3	195	Ã
010C	00	000	
010D	00	000	
010E	00	000	

MOV AX, 00005h
MOV BX, 0000Ah
ADD AX, BX
SUB AX, 00001h
RET
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL

User Screen Actual Source ALU Stack FLAGS

Stack

0B56:FFFE	0000
0B56:FFFC	0000
0B56:FFFA	0000
0B56:FFF8	0000
0B56:FFF6	0000
0B56:FFF4	0000
0B56:FFF2	0000
0B56:FFF0	0000
0B56:FFEE	0000
0B56:FFEC	0000
0B56:FFEA	0000
0B56:FFE8	0000
0B56:FFE6	0000
0B56:FFE4	0000
0B56:FFE2	0000
0B56:FFE0	0000
0B56:FFDE	0000
0B56:FFDC	0000

Actual Emulated Source...

```
01 org 100h
02
03 MOV AX, 5
04 MOV BX, 10
05
06 ; 5 + 10 = 15 (000Fh)
07 ADD AX, BX
08
09 ; 15 - 1 = 14 (000Eh)
10 SUB AX, 1
11
12 ret
13
14
15
```

ALU - Arithmetic & Logical Unit

F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0





Exécution

Exécution de la deuxième instruction

- Nous appuyons sur le bouton "Single Step" pour exécuter la deuxième instruction (MOV BX, 10).
- A l'issue de cette exécution, nous constatons les changements suivants :
 - Le registre BX contient la valeur 10 ;
 - Le registre IP contient la valeur 106h (IP passe de 103h à 106h car MOV BX, 10 est codé sur 3 octets) ;
- La prochaine instruction pointée par IP (ADD AX, BX) est surlignée dans les différents écrans.





Exécution

Exécution de la deuxième instruction

Emulator: noname.com - 8086 Microprocessor Emulator

File Math Debug View Virtual Devices Virtual Drive Help

Load Reload Single Step Run step delay ms: 0

Registers

	H	L
AX	00	05
BX	00	0A
CX	00	0C
DX	00	00
CS	0B56	
IP	0106	
SS	0B56	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0B56	
ES	0B56	

memory (1K) at: 0B56 : 0100 0B56 : 0100

Disassemble:

Offset:	Hex	Dec	ASCII
0100:	B8	184	.
0101:	05	005	■
0102:	00	000	
0103:	BB	187	>>
0104:	0A	010	■
0105:	00	000	
0106:	03	003	■
0107:	03	195	A
0108:	2D	045	-
0109:	01	001	■
010A:	00	000	
010B:	C3	195	ã
010C:	00	000	
010D:	00	000	
010E:	00	000	

MOV AX, 00005h
MOV BX, 0000Ah
ADD AX, BX
SUB AX, 00001h
RET
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL

User Screen Actual Source ALU Stack FLAGS

Stack

0B56:FFFE	0000
0B56:FFFC	0000
0B56:FFFA	0000
0B56:FFF8	0000
0B56:FFF6	0000
0B56:FFF4	0000
0B56:FFF2	0000
0B56:FFF0	0000
0B56:FFEE	0000
0B56:FFEC	0000
0B56:FFEA	0000
0B56:FFE8	0000
0B56:FFE6	0000
0B56:FFE4	0000
0B56:FFE2	0000
0B56:FFE0	0000
0B56:FFDE	0000
0B56:FFDC	0000

Actual Emulated Source...

```
01 org 100h
02
03 MOV AX, 5
04 MOV BX, 10
05
06 ; 5 + 10 = 15 (000Fh)
07 ADD AX, BX
08
09 ; 15 - 1 = 14 (000Eh)
10 SUB AX, 1
11
12 ret
13
14
15
```

ALU - Arithmetic & Logical Unit

F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0





Exécution

Exécution de la troisième instruction

- Nous exécutons la troisième instruction (ADD AX, BX), ce qui entraîne les changements suivants :
 - La fenêtre ALU affiche le contenu des deux registres temporaires d'entrées et le résultat de l'opération ;
 - Le registre AX contient la valeur 0Fh ($05h + 0Ah = Fh$) ;
 - Le registre IP contient la valeur 108h (IP passe de 106h à 108h car ADD AX, BX est codé sur 2 octets) ;
- La prochaine instruction pointée par IP (SUB AX, 1) est surlignée dans les différents écrans.





Exécution

Exécution de la troisième instruction

Emulator: noname.com - 8086 Microprocessor Emulator

File Math Debug View Virtual Devices Virtual Drive Help

Load Reload Single Step Run step delay ms: 0

Registers

	H	L
AX	00	0F
BX	00	0A
CX	00	0C
DX	00	00
CS	0B56	
IP	0108	
SS	0B56	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0B56	
ES	0B56	

memory (1K) at: 0B56 : 0100

Offset	Hex	Dec	ASCII
0100	B8	184	
0101	05	005	
0102	00	000	
0103	BB	187	>>
0104	0A	010	
0105	00	000	
0106	03	003	
0107	C3	195	ã
0108	2D	045	-
0109	01	001	
010A	00	000	
010B	C3	195	ã
010C	00	000	
010D	00	000	
010E	00	000	

Disassemble:

```
MOV AX, 00005h
MOV BX, 0000Ah
ADD AX, BX
SUB AX, 00001h
RET
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
```

Stack

Address	Value
0B56:FFFE	0000
0B56:FFFC	0000
0B56:FFFA	0000
0B56:FFF8	0000
0B56:FFF6	0000
0B56:FFF4	0000
0B56:FFF2	0000
0B56:FFF0	0000
0B56:FFEE	0000
0B56:FFEC	0000
0B56:FFEA	0000
0B56:FFE8	0000
0B56:FFE6	0000
0B56:FFE4	0000
0B56:FFE2	0000
0B56:FFE0	0000
0B56:FFDE	0000
0B56:FFDC	0000

Actual Emulated Source...

```
01 org 100h
02
03 MOV AX, 5
04 MOV BX, 10
05
06 ; 5 + 10 = 15 (000Fh)
07 ADD AX, BX
08
09 ; 15 - 1 = 14 (000Eh)
10 SUB AX, 1
11
12 ret
13
14
15
```

ALU - Arithmetic & Logical Unit

F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1



Exécution

Exécution de la quatrième instruction

- Nous exécutons la quatrième instruction (SUB AX, 1), ce qui entraîne les changements suivants :
 - La fenêtre ALU affiche le contenu des deux registres temporaires d'entrées et le résultat de l'opération ;
 - Le registre AX contient la valeur 0Fh ($0Fh + 01h = Eh$) ;
 - Le registre IP contient la valeur 10Bh (IP passe de 108h à 10Bh car SUB AX, 1 est codé sur 3 octets) ;
- La prochaine instruction pointée par IP (RET) est surlignée dans les différents écrans.





Exécution

Exécution de la quatrième instruction

The screenshot displays the Emu8086 emulator interface with several windows open. The main window shows the registers, memory, and disassembly. The registers window highlights the AX register with a value of 000E and the IP register with a value of 010B. The memory window shows the current instruction at offset 010B: C3 195 A. The disassembly window shows the instruction RET. The stack window shows the stack pointer at 0B56:FFFE. The actual emulated source code window shows the instruction RET. The ALU window shows the ALU register with a value of 0000.

Registers:

Register	H	L
AX	00	0E
BX	00	0A
CX	00	0C
DX	00	00
CS	0B56	
IP	010B	
SS	0B56	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0B56	
ES	0B56	

Memory (1K) at:

Offset	Hex	Dec	ASCII
0100	B8	184	
0101	05	005	
0102	00	000	
0103	BB	187	
0104	0A	010	
0105	00	000	
0106	03	003	
0107	C3	195	A
0108	2D	045	-
0109	01	001	
010A	00	000	
010B	C3	195	A
010C	00	000	
010D	00	000	
010E	00	000	

Disassembly:

```
MOV AX, 00005h
MOV BX, 0000Ah
ADD AX, BX
SUB AX, 00001h
RET
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
```

Stack:

Address	Value
0B56:FFFE	0000
0B56:FFFC	0000
0B56:FFFA	0000
0B56:FFF8	0000
0B56:FFF6	0000
0B56:FFF4	0000
0B56:FFF2	0000
0B56:FFF0	0000
0B56:FFEE	0000
0B56:FFEC	0000
0B56:FFEA	0000
0B56:FFE8	0000
0B56:FFE6	0000
0B56:FFE4	0000
0B56:FFE2	0000
0B56:FFE0	0000
0B56:FFDE	0000
0B56:FFDC	0000

Actual Emulated Source...

```
01 org 100h
02
03 MOV AX, 5
04 MOV BX, 10
05
06 ; 5 + 10 = 15 (000Fh)
07 ADD AX, BX
08
09 ; 15 - 1 = 14 (000Eh)
10 SUB AX, 1
11
12 ret
13
14
15
```

ALU - Arithmetic & Logical Unit

F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0



Exécution

Exécution de la cinquième instruction

■ Nous exécutons la cinquième instruction (RET). Comme nous le verrons un peu plus loin, cette instruction est équivalente dans ce cas de figure à effectuer POP IP ce qui amène les deux traitements ci-dessous :

- On dépile la valeur stockée dans la pile à la position SS:SP (autrement la case mémoire dont l'adresse est $SS * 16 + SP$) pour la charger dans le registre IP. Dans notre exemple, SS:SP vaut 0B56h:FFFEh et **le mot** (autrement dit les 2 octets consécutifs) stockée à cette adresse valent 0000h.

A l'issue de ce traitement, IP est donc égale à 0000h et la prochaine instruction qui sera exécutée est située à l'adresse CS:IP soit 0B56h:0000h





Exécution

Exécution de la cinquième instruction

- Nous exécutons la cinquième instruction (RET). Comme nous le verrons un peu plus loin, cette instruction est équivalente dans ce cas de figure à effectuer POP IP ce qui amène les deux traitements ci-dessous :
 - Dans le même temps, on modifie la valeur de SP car on a dépilé un élément : on ajoute 2 à SP donc SP vaut $\text{FFFEh} + 0002\text{h} = 10000\text{h}$ soit 0000h car SP est un registre 16 bits





Exécution

Exécution de la cinquième instruction

Emulator: noname.com - 8086 Microprocessor Emulator

File Math Debug View Virtual Devices Virtual Drive Help

Load Reload Single Step Run step delay ms: 0

Registers

	H	L
AX	00	0E
BX	00	0A
CX	00	0C
DX	00	00
CS	0B56	
IP	0000	
SS	0B56	
SP	0000	
BP	0000	
SI	0000	
DI	0000	
DS	0B56	
ES	0B56	

memory (1K) at: 0B56 : 0100 0B56 : 0000

Disassemble:

Offset	Hex	Dec	ASCII	Instruction
0100	B8	184		ADD [BX + SI], AL
0101	05	005		ADD [BX + SI], AL
0102	00	000		ADD [BX + SI], AL
0103	B8	187		ADD [BX + SI], AL
0104	0A	010		ADD [BX + SI], AL
0105	00	000		ADD [BX + SI], AL
0106	03	003		ADD [BX + SI], AL
0107	C3	195	Ã	ADD [BX + SI], AL
0108	2D	045	-	ADD [BX + SI], AL
0109	01	001		ADD [BX + SI], AL
010A	00	000		ADD [BX + SI], AL
010B	C3	195	Ã	ADD [BX + SI], AL
010C	00	000		ADD [BX + SI], AL
010D	00	000		ADD [BX + SI], AL
010E	00	000		ADD [BX + SI], AL

User Screen Actual Source ALU Stack FLAGS

Stack

0B56:0020	0000
0B56:001E	0000
0B56:001C	0000
0B56:001A	0000
0B56:0018	0000
0B56:0016	0000
0B56:0014	0000
0B56:0012	0000
0B56:0010	0000
0B56:000E	0000
0B56:000C	0000
0B56:000A	0000
0B56:0008	0000
0B56:0006	0000
0B56:0004	0000
0B56:0002	0000
0B56:0000	20CD

Actual Emulated Source...

```
01 org 100h
02
03 MOV AX, 5
04 MOV BX, 10
05
06 ; 5 + 10 = 15 (000Fh)
07 ADD AX, BX
08
09 ; 15 - 1 = 14 (000Eh)
10 SUB AX, 1
11
12 ret
13
14
15
```

ALU - Arithmetic & Logical Unit

F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0





Exécution

Exécution de la sixième instruction

- Nous appuyons sur "single step" et nous exécutons une instruction que nous n'avons pas saisie dans notre programme (INT 020h) qui a été automatiquement ajoutée par le système d'exploitation (de même que la valeur qui était empilée et la valeur des registres SS et SP).
- Nous décrirons le mécanisme des interruptions logicielles un peu plus loin. A notre niveau, nous devons comprendre que INT 020h donne l'ordre au processeur d'exécuter l'interruption (comprendre la routine ou la procédure) numéro 20h. A l'appel de cette procédure, le processeur modifie le contenu de ses registres car il saute dans une autre zone mémoire puis il exécute cette instruction qui consiste simplement à terminer le programme et à rendre la main au système d'exploitation (ici une émulation du DOS)





Exécution

Exécution de la sixième instruction

Emulator: noname.com - 8086 Microprocessor Emulator

File Math Debug View Virtual Devices Virtual Drive Help

Load Reload Single Step Run step delay ms: 0

Registers

	H	L
AX	00	0E
BX	00	0A
CX	00	0C
DX	00	00
CS	F400	
IP	0150	
SS	0B56	
SP	FFFA	
BP	0000	
SI	0000	
DI	0000	
DS	0B56	
ES	0B56	

memory (1K) at: 0B56 : 0100 F400 : 0150

Disassemble:

Offset	Hex	Dec	ASCII
0100	B8	184	
0101	05	005	
0102	00	000	
0103	BB	187	
0104	0A	010	
0105	00	000	
0106	03	003	
0107	C3	195	Å
0108	2D	045	-
0109	01	001	
010A	00	000	
010B	C3	195	Å
010C	00	000	
010D	00	000	
010E	00	000	

BIOS DI
INT 020h
IRET

ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
SBB CL, BH
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL

User Screen Actual Source ALU Stack FLAGS

Stack

0B56:FFFA	0002
0B56:FFF8	0000
0B56:FFF6	0000
0B56:FFF4	0000
0B56:FFF2	0000
0B56:FFF0	0000
0B56:FFEE	0000
0B56:FFEC	0000
0B56:FFEA	0000
0B56:FFE8	0000
0B56:FFE6	0000
0B56:FFE4	0000
0B56:FFE2	0000
0B56:FFE0	0000
0B56:FFDE	0000
0B56:FFDC	0000
0B56:FFDA	0000
0B56:FFD8	0000

Actual Emulated Source...

```
01 org 100h
02
03 MOV AX, 5
04 MOV BX, 10
05
06 ; 5 + 10 = 15 (000Fh)
07 ADD AX, BX
08
09 ; 15 - 1 = 14 (000Eh)
10 SUB AX, 1
11
12 ret
13
14
15
```

ALU - Arithmetic & Logical Unit

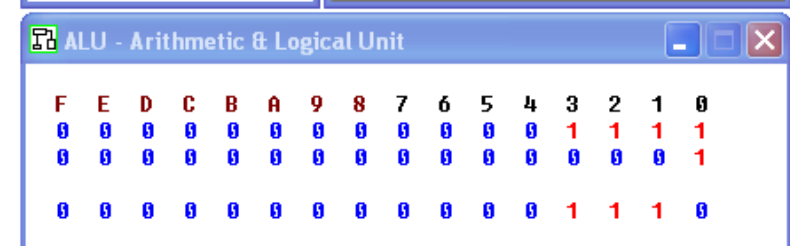
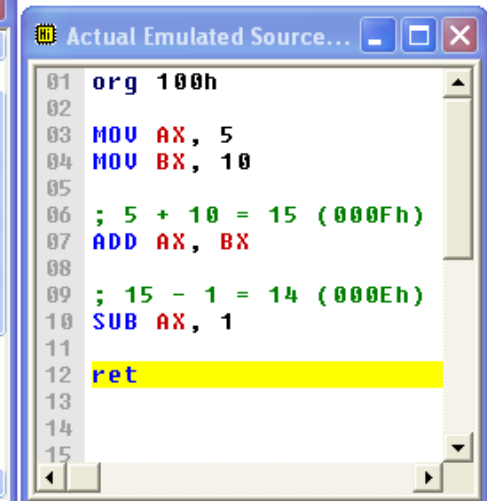
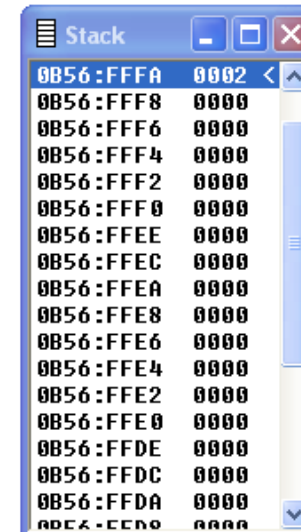
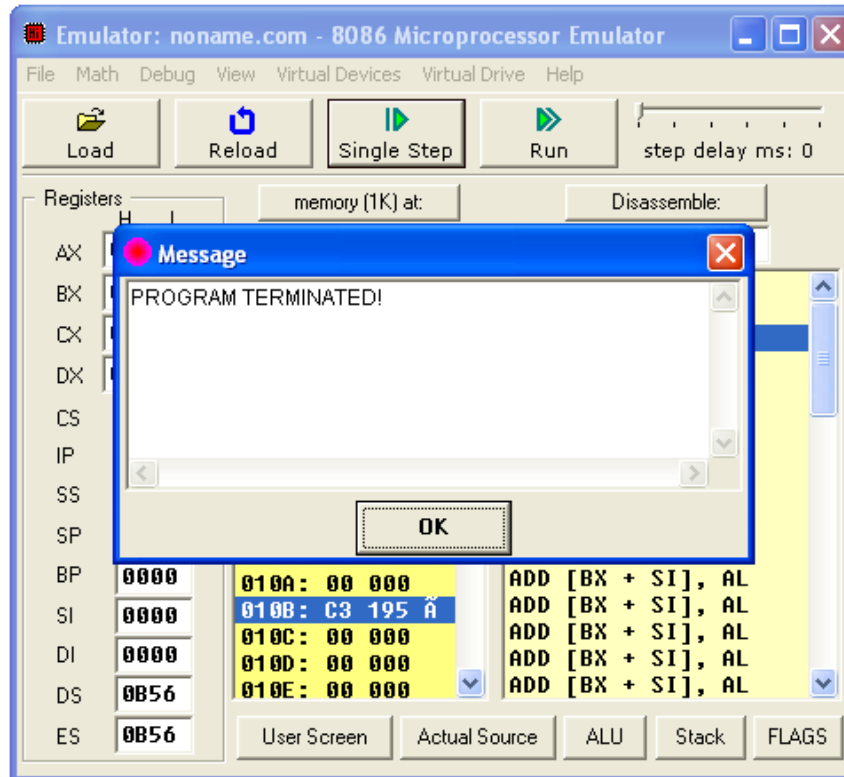
F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0





Exécution

Exécution de la sixième instruction





Pause-réflexion sur cette 2^{ème} partie

Avez-vous des questions ?





Manipulation de base



Plan de la partie

Voici les chapitres que nous allons aborder :

- Opérations logiques.
- Opérations arithmétiques.
- Utilisation du bit CF.
- Multiplication et division.
- Adressage direct.
- Adressage par registre.





Opérations logiques

AND, OR, NOT et XOR

- Le 8086 permet naturellement d'effectuer des opérations binaires classiques grâce aux mnémoniques **AND**, **OR**, **NOT** et **XOR**.

AND registre, nombre	ou	AND registre, registre
OR registre, nombre	ou	OR registre, registre
NOT registre, nombre	ou	NOT registre, registre
XOR registre, nombre	ou	XOR registre, registre

- Exemple :

```
MOV AL, 01100001b
AND AL, 11011111b ; On obtient 01000001b
OR  AL, 01110110b ; On obtient 01110111b
NOT AL           ; On obtient 10001000b
XOR AL, 01010101b ; On obtient 11011101b
```





Opérations logiques

Décalage à gauche et décalage à droite

- Le 8086 permet d'effectuer des opérations de décalage sur des nombres signés ou non signés. Les bits qui sont expulsés sont stockés dans le bit CF :
 - **SHL** (*shift left*) et **SAL** (*shift arithmetic left*) ;
 - **SHR** (*shift right*) et **SAR** (*shift arithmetic right*).
- La différence entre les deux types de décalage est que dans la seconde version le bit de signe n'est pas concerné par ce décalage.
- Cette opération est plus rapide à exécuter que l'opération de multiplication ou de division donc il est préférable d'utiliser un décalage pour effectuer une division ou une multiplication par 2.





Opérations logiques

Décalage à gauche et décalage à droite

- Le nombre de bits de décalage est indiqué dans la seconde opérande.
- La syntaxe d'utilisation est la suivante :

```
SHL/SAL/SHR/SAR registre, nombre de bits de décalage
```

- Exemple :

```
MOV AL, 11110000b
SHL AL, 1 ; On obtient 11100000b
SAL AL, 1 ; On obtient 11000000b
SAR AL, 1 ; On obtient 10100000b
SHR AL, 1 ; On obtient 01010000b
```





Opérations logiques

Rotation à gauche et rotation à droite

- Le 8086 permet également de réaliser des opérations de rotation sur des nombres signés ou non signés. Les bits qui sont expulsés sont remplacés dans les trous formés par ce décalage :
 - **ROL** (*rotate left*) et **RCL** (*rotate throw carry left*) ;
 - **ROR** (*rotate right*) et **RCR** (*rotate throw carry right*).
- La différence entre les deux types de décalage est que dans la seconde version le bit CF est utilisé en plus des bits à modifier (lors d'un mouvement, le trou est rempli avec une copie du bit CF et l'autre bit, qui est expulsé, est placé dans CF).





Opérations logiques

Rotation à gauche et rotation à droite

- Le nombre de bits de décalage est indiqué dans la seconde opérande.
- La syntaxe d'utilisation est la suivante :

```
ROL/RCL/ROR/RCR registre, nombre de bits de décalage
```

- Exemple :

```
MOV AL, 11110000b
ROL AL, 1 ; On obtient 11100001b (on suppose CF=0)
RCL AL, 1 ; On obtient 11000010b et CF = 1
RCR AL, 1 ; On obtient 11100001b et CF = 0
ROR AL, 1 ; On obtient 11110000b
```





Opérations arithmétiques

Addition et soustraction sans retenue

- Nous avons vu dans la partie précédente que les mnémoniques ADD (*addition*) et SUB (*subtraction*) permettent d'effectuer des additions et des soustractions entre deux registres ou entre un registre et un nombre.
 - ADD AX, BX effectue l'opération $AX \leftarrow AX + BX$
 - SUB AX, BX effectue l'opération $AX \leftarrow AX - BX$
- Ces opérations sont effectuées sans tenir compte de l'état initial du bit CF du registre d'état mais l'état de ce bit peut être modifié à l'issue de l'opération.





Opérations arithmétiques

Incrémentation et décrémentation

- Le 8086 propose deux instructions qui permettent d'incrémenter ou de décrémentation la valeur d'un registre :
 - **INC** AX incrémente la valeur stockée dans AX
 - **DEC** BX décrémente la valeur stockée dans BX
- INC AX donne le même résultat que ADD AX, 1 mais le premier mnémonique correspond à un opcode de 1 octet alors que l'opcode correspondant au second mnémonique en occupe 3.
- On constate la même chose si on compare DEC BX et SUB BX, 1





Utilisation du bit CF

L'addition sur 32 bits

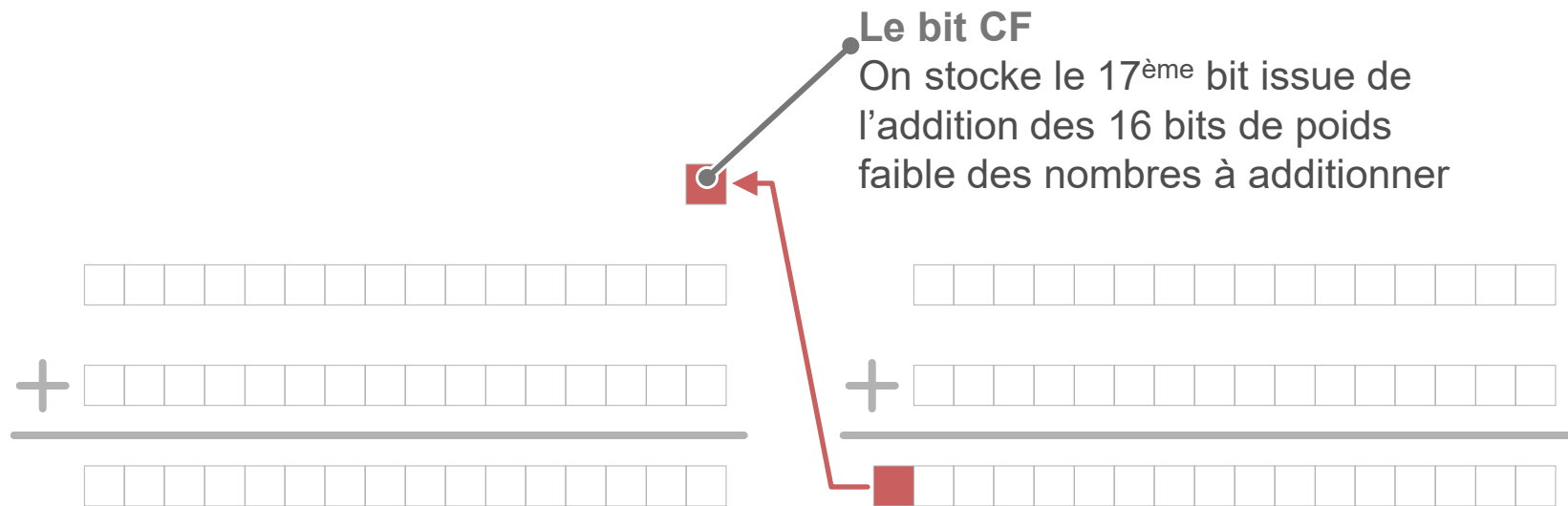
- Dans certains cas, il peut être nécessaire d'effectuer des additions sur 32 bits.
- Cependant, la taille des registres est limitée à 16 bits sur le 8086, nous devons donc scinder les nombres codés sur 32 bits en deux parties :
 - 1 registre pour stocker les 16 bits de poids fort ;
 - 1 registre pour stocker les 16 bits de poids faible.
- Lorsqu'on effectue une addition sur 32 bits, on effectue d'abord l'addition entre les 16 bits de poids faible puis l'addition entre les 16 bits de poids fort en utilisant le bit CF pour propager la retenue entre les deux blocs de 16 bits.





Utilisation du bit CF

L'addition sur 32 bits





Utilisation du bit CF

Les mnémoniques ADC et SBB

- Pour demander au processeur d'effectuer des additions qui tiennent compte du bit CF, nous devons utiliser une nouvelle mnémonique : **ADC** (*addition with carry*).
- Il n'existe pas de mnémonique SUBC mais un mnémonique appelé **SBB** (*subtract with borrow*) qui effectue le même type d'opération mais en sens inverse.
- Comme les opérations ADC et SBB modifient l'état du bit CF, on peut donc s'en servir pour effectuer des opérations sur 48 bits, 64 bits ... en stockant les résultats intermédiaire en mémoire : l'opération se fera donc par bloc de 16 bits.





Utilisation du bit CF

L'addition sur 32 bits grâce au mnémonique ADC

- Construire un programme assembleur qui effectue les traitements suivants :
 - chargement du nombre 10F0C0BBh sur les registres AX et BX (le registre AX contient le mot de poids fort) ;
 - chargement du nombre 0A10DDFFh sur les registres CX et DX (le registre CX contient le mot de poids fort) ;
 - addition de 10F0C0BBh et 0A10DDFFh en utilisant le mécanisme décrit précédemment.
 - retour vers le programme appelant (l'OS).





Utilisation du bit CF

Les mnémoniques STC, CLC et CMC

- Le 8086 offre 3 instructions qui permettent de modifier directement l'état du bit CF :
 - **STC** (**s**et **c**arry flag) permet de mettre CF à 1 ;
 - **CLC** (**c**lear **c**arry flag) permet de mettre CF à 0 ;
 - **CMC** (**c**omplement **c**arry flag) permet d'inverser l'état du bit CF.





Utilisation du bit CF

Les mnémoniques CLD, CLI, STD et STI

- Il est à remarquer que le 8086 offre 4 autres instructions qui permettent de modifier directement l'état des bits DF (direction) et IF (interruptions externes) :
 - **STD** (set direction flag) permet de mettre DF à 1 ;
 - **CLD** (clear direction flag) permet de mettre DF à 0 ;
 - **STI** (set interrupt flag) permet de mettre IF à 1 ;
 - **CLI** (clear interrupt flag) permet de mettre IF à 0.
- Les mnémoniques STI et CLI sont utiles pour implanter au niveau assembleur des procédures qui ne peuvent pas être interrompues par des événements externes (cf. cours de théorie des systèmes d'exploitation et module 7).





Utilisation du bit CF

Utilisation des mnémoniques STC, CLC et CMC

- Modifier le programme précédent :
 - placer l'instruction STC avant le premier ADC et l'instruction CLC entre les deux ADC ;
 - observer l'état du bit CF lorsqu'on exécute ces nouvelles instructions.

- Modifier un nouvelle fois le programme :
 - modifier le programme de manière à remplacer les instructions STC et CLC par CMC ;
 - observer l'état du bit CF lorsqu'on exécute ces nouvelles instructions.





Multiplication et division

La multiplication non signée

- Le 8086 utilise le mnémonique **MUL** pour effectuer des multiplications mais son mode de fonctionnement est très différent par rapport à celui de l'addition et la soustraction.
- Si nous voulons multiplier un nombre N par un nombre M, nous devons d'abord copier le nombre N dans le registre AL (ou AX) puis invoquer l'instruction MUL M :
 - si l'opérande est un octet, on calcule **AL * opérande** et le résultat est stocké dans **AX** ;
 - si l'opérande est un mot, on calcule **AX * opérande** et le résultat est stocké dans les registres **DX** et **AX** (DX contenant la partie de poids fort).





Multiplication et division

La multiplication signée

- Le 8086 propose un autre mnémonique pour effectuer les multiplication entre deux nombres signées.
- Le mnémonique **IMUL** est utilisé de la même manière que le mnémonique MUL à la différence que les opérandes sont considérés comme des nombres signés (codés en complément à 2) et que le résultat est un nombre signé.





Multiplication et division

La division euclidienne non signée

- Le mnémonique permettant d'effectuer une division euclidienne est DIV et son mode de fonctionnement est voisin de celui de MUL.
- Si nous voulons diviser un nombre N par un nombre M, nous devons d'abord copier le nombre N dans le registre AX (ou AX et DX) puis invoquer l'instruction DIV M :
 - si l'opérande est un octet, on calcule **AX / opérande** : le quotient est stocké dans **AL** et le reste dans **AH** ;
 - dans le cas d'un mot, on calcule **(AX DX) / opérande** : le quotient est stocké dans **AX** et le reste dans **DX**.





Multiplication et division

La division euclidienne signée

- A l'instar de la multiplication signée, le 8086 propose un autre mnémonique pour effectuer les divisions euclidiennes entre deux nombres signés.
- Le mnémonique **IDIV** est donc utilisé de la même manière que le mnémonique DIV : les différences sont que les opérandes sont considérées comme des nombres signés (codés en complément à 2) et que le résultat est un nombre signé.





Multiplication et division

Exercice

- Ecrire des programmes assembleurs qui effectuent les traitements suivants :
 - la multiplication 15×-20 sur 8 bits ;
 - la multiplication 115×-10 sur 8 bits ;
 - la multiplication 115×-10 sur 16 bits ;
 - la division $93 / 3$ sur 8 bits ;
 - la division $520 / -10$ sur 8 bits ;
 - la division $520 / -10$ bits sur 16 bits.

- Que constate-t-on au niveau du registre d'état pour les calculs 2 et 5 par rapport aux autres calculs ?





L'adressage

Adressage immédiat et adressage par registre

- Jusqu'à présent, nous avons rencontré deux modes d'adressages :
 - Le mode **immédiat** où un opérande est un nombre codé sur 8 ou 16 bits directement dans l'opcode.
Exemple : MOV AX, 10
 - Le mode **par registre** qui fait intervenir deux registres.
Exemple : MOV AX, BX
- Ces deux modes **ne faisait pas intervenir la mémoire centrale**, mais nous allons maintenant introduire un autre mode qui manipule les données en mémoire.





L'adressage

Rappel sur l'adressage des instructions dans CS

- Nous avons appris dans la deuxième partie que les instructions d'un programme sont pointées par le registre IP qui contient une adresse relative au segment de code (*Code Segment* ou CS).
- L'adresse de base est stockée dans CS.
- L'adresse absolue de l'instruction est donc CS:IP qui est égal à $CS \times 16 + IP$.





L'adressage

L'adressage des données dans DS

- L'adressage des données dans le segment de données (Data Segment ou DS) peut s'effectuer de différentes manières (en utilisant différentes combinaisons) :
 - DS : DEP
 - DS : BX
 - DS : SI
 - DS : DI
 - DS : BX+DEP
 - DS : SI+DEP
 - DS : DI+DEP
 - DS : BX+SI
 - DS : BX+DI
 - DS : BX+SI+DEP
 - DS : BX+DI+DEP
 - DEP est un nombre qui correspond à un décalage en mémoire, exprimé en octets





L'adressage

L'instruction MOV

- D'un point de vue programmation, cet adressage des données peut être utilisé à l'aide de l'instruction **MOV**.
- D'un point de vue syntaxique, on distingue ce type d'adressage de ceux évoqués précédemment grâce à l'utilisation des crochets [] qui entoure la seconde partie de l'adresse
- Exemples :
 - « **MOV DS:[BX], 10** » signifie qu'on stocker 10 dans la case dont l'adresse est DS:BX (soit $DS \times 16 + BX$).
 - « **MOV AX, DS:[BX+DI+0x10h]** » signifie qu'il faut copier le contenu de la case mémoire d'adresse DS:BX+DI+0x10h dans le registre AX.





L'adressage

L'adressage des données dans SS et ES

- On peut également adresser des données dans la pile (Stack Segment ou SS) ou dans le segment supplémentaire (Extra Segment ou ES). On a alors les combinaisons supplémentaires suivantes :

- ES : DEP

- ES : BX+DI

- ES : BX

- ES : BX+SI+DEP

- ES : SI

- ES : BX+DI+DEP

- ES : DI

- SS : BP+DEP

- ES : BX+DEP

- SS : BP+SI

- ES : SI+DEP

- SS : BP+DI

- ES : DI+DEP

- SS : BP+SI+DEP

- ES : BX+SI

- SS : BP+DI+DEP





L'adressage

L'instruction MOV – Remarque complémentaire

- L'adressage des données dans ces segment s'effectue de la même manière que précédemment :
 - « **MOV ES:[BX+SI], DX** » permet de stocker le contenu du registre DX dans la case mémoire d'adresse ES:[BX+SI].
 - « **MOV CX, SS:[BP+0x02h]** » permet de copier, dans le registre CX, le contenu dans la case mémoire d'adresse SS:[BP+0x02h].
- Si le segment concerné est DS, il n'est pas nécessaire de l'indiquer car il est sous-entendu :

MOV [BX], 4 est équivalent à MOV DS:[BX], 4





L'adressage

Adressage en mode mémoire avec ADD, SUB et ADC

- Jusqu'à présent, nous avons utilisé les instructions ADD, SUB et ADC en respectant l'une des deux syntaxes ci-dessous (qui correspondent respectivement au mode immédiat et au mode par registre) :
 - ADD/SUB/ADC registre, immédiat
 - ADD/SUB/ADC registre, registre
- Ces instructions peuvent également s'appliquer en mode d'adressage en mémoire :
 - ADD/SUB/ADC registre, mémoire
 - ADD/SUB/ADC mémoire, registre
 - ADD/SUB/ADC mémoire, immédiat





L'adressage

Adressage en mode mémoire avec MUL, IMUL, DIV et IDIV

- Les instructions MUL, IMUL, DIV et IDIV que nous avons évoqués précédemment ne supportent pas le mode immédiat.
- Ces instructions s'utilisent uniquement en mode par registre et en mode en mémoire. Nous avons donc les syntaxes suivantes :
 - MUL/IMUL/DIV/IDIV registre
 - MUL/IMUL/DIV/IDIV mémoire





L'adressage

Spécification de la taille des données

- Il peut être utile de spécifier si le pointeur manipule un mot ou un octet (cela est particulièrement utile pour opération de division et de multiplication).
- Pour cela, nous devons ajouter une information supplémentaire devant le pointeur :
 - Si le pointeur manipule un mot, nous le faisons précéder de « **WORD PTR** » ou « **w.** » :

Exemple : **MOV WORD PTR ES:[BX+DI+0x10h], AX**

- Dans le cas contraire, nous le faisons précéder de « **BYTE PTR** » ou « **b.** » :

Exemple : **DIV b.ES:[BX+DI+0x10h]**





L'adressage

Exercice

- Construire un programme effectuant les tâches suivantes :
 - chargement de l'octet 0x10h dans la case mémoire DS:BX en mettant BX au préalable à 0x200h (on ne touche pas à DS) ;
 - chargement du nombre 0x03h dans le registre AX ;
 - division du contenu du registre AX par celui de la case mémoire DS:BX en utilisant la fonction DIV ;
 - multiplication du contenu du registre AX par celui de la case mémoire DS:BX en utilisant la fonction MUL ;
 - copie du résultat dans la case mémoire située après celle(s) qui stocke(nt) le nombre de départ (attention à ne pas l'écraser).





L'adressage

Exercice (suite)

- Une fois le programme implanté, effectuer le travail suivant :
 - lancer le simulateur et indiquer dans un tableau quel est le contenu des registres et des éléments mémoire impliqués dans le programme ;
 - modifier le programme de manière à ce que 0x10h soit considéré comme un mot. Relancer alors le simulateur et reconstruire le tableau ;
 - modifier le programme de manière à manipuler le nombre -0x10h sous forme d'octet. Relancer alors le simulateur et reconstruire le tableau ;
 - modifier le programme de manière à manipuler le nombre -0x10h sous forme de mot. Relancer alors le simulateur et reconstruire le tableau.





Pause-réflexion sur cette 3^{ème} partie

Avez-vous des questions ?





Les sauts



Plan de la partie

Voici les chapitres que nous allons aborder :

- La notion d'étiquette.
- Les sauts inconditionnels.
- La comparaison.
- Les sauts conditionnels.
- Les boucles.





La notion d'étiquette

Le principe

- L'étiquette (*label* en anglais) est utilisée par les mnémoniques de contrôle du flot d'instructions comme JMP, JO, JNZ ...
- Elle permet de spécifier au programme à quel endroit continuer l'exécution.
- Cette étiquette est **convertie en une adresse** au moment de l'assemblage des instructions de saut.





La notion d'étiquette

La déclaration d'un label

- Pour ajouter un label dans le programme, il suffit de mettre un nom (qui ne commence pas par un chiffre) suivi de « : ».

```
eti1: MOV AX, 10  
      ADD AX, 20  
      INC AX  
  
eti2: MOV BX, 20  
      SUB BX, 30  
      DEC BX
```





Les sauts inconditionnels

Le mnémonique JMP

- Le branchement inconditionnel consiste simplement à sauter d'une position dans le code à une autre position pour continuer l'exécution.
- Ce type de branchement est opposé aux branchements conditionnels (étudiés plus loin) qui réalisent des sauts en fonction de test effectués sur les bits du registre d'état.
- L'instruction utilisée pour réaliser le saut inconditionnel est JMP suivi d'un nom de l'étiquette (ou d'un nombre représentant l'adresse de destination codée sur 4 octets mais ce n'est pas conseillé).





Les sauts inconditionnels

Exemple

```
ORG 100h

MOV AX, 5    ; assigner 5 à AX.
MOV BX, 2    ; assigner 2 à BX.

JMP calc     ; aller à "calc".

back: JMP stop ; aller à "stop".

calc: ADD AX, BX ; ajouter BX à AX.

      JMP back  ; aller à "back".

stop: RET      ; redonner la main à l'OS.

END
```





Les sauts inconditionnels

Exercice

- Etant donné le programme incomplet suivant :

```
ORG 100h

...
INC AX
...
DEC AX
...
MOV AX, 10
...
INC AX
...
DEC AX
...

END
```





Exercise

-
- ```

graph TD
 DEBUT([DEBUT]) -- blue --> op4([op4: INC AX])
 op4 -- blue --> op3([op3: DEC AX])
 op3 -- blue --> op4
 op3 -- green --> op1([op1: MOV AX, 10])
 op1 -- green --> op2([op2: INC AX])
 op2 -- green --> op5([op5: DEC AX])
 op5 -- green --> op3
 DEBUT -.-> R1[]
 op1 -.-> R2[]
 style R1 fill:none,stroke:red,stroke-dasharray: 5 5
 style R2 fill:none,stroke:red,stroke-dasharray: 5 5

```





# La comparaison

## Le mnémonique CMP

- La comparaison de deux nombres s'effectue en faisant une pseudo-soustraction qui n'affecte les drapeaux ci-dessous dans le registre d'état :
  - OF : Overflow Flag ;
  - SF : Sign Flag ;
  - ZF : Zero Flag ;
  - AF : Auxiliary Carry Flag ;
  - PF : Parity Flag ;
  - CF : Carry Flag.





# La comparaison

## Les différents types de comparaison

- Compte tenu de la « ressemblance » avec l'instruction SUB, les types des opérandes traitées sont les mêmes, on peut donc effectuer les comparaisons suivantes :
  - CMP registre, mémoire
  - CMP mémoire, registre
  - CMP registre, registre
  - CMP mémoire, immédiat
  - CMP registre, immédiat





# Les sauts conditionnels

## Le principe

- L'instruction CMP que nous avons décrit juste avant ainsi que d'autres instructions qui affectent les drapeaux du registre d'état sont souvent utilisées avec des instructions de branchement conditionnel afin d'implanter des boucles, des tests ...
- A l'instar de l'instruction JMP, les instructions de branchement conditionnel s'utilisent avec un label. Elles doivent être placées juste après l'instruction (CMP, DEC, INC ...) qui modifie l'état du registre d'état sinon les autres mnémoniques placées entre cette instruction et le branchement conditionnel pourraient altérer les bits du registre d'état.





## Les sauts conditionnels

### Les mnémoniques de sauts conditionnels

| Instruction | Description                              | Condition              | Instruction opposée |
|-------------|------------------------------------------|------------------------|---------------------|
| JA          | Jump if <b>A</b> bove                    | CF = 0 et ZF = 0       | JNA                 |
| JAE         | Jump if <b>A</b> bove or <b>E</b> qual   | CF = 0                 | JNAE                |
| JB          | Jump if <b>B</b> elow                    | CF = 1                 | JNB                 |
| JBE         | Jump if <b>B</b> elow or <b>E</b> qual   | CF = 1 ou ZF = 1       | JNBE                |
| JC          | Jump if <b>C</b> arry                    | CF = 1                 | JNC                 |
| JE          | Jump if <b>E</b> qual                    | ZF = 1                 | JNE                 |
| JG          | Jump if <b>G</b> reater                  | ZF = 0 et SF = OF      | JNG                 |
| JGE         | Jump if <b>G</b> reater or <b>E</b> qual | SF = OF                | JNGE                |
| JL          | Jump if <b>L</b> ess                     | SF $\neq$ OF           | JNL                 |
| JLE         | Jump if <b>L</b> ess or <b>E</b> qual    | ZF = 1 ou SF $\neq$ OF | JNLE                |







## Les sauts conditionnels

### Les mnémoniques de sauts conditionnels

| Instruction | Description                                          | Condition         | Instruction opposée |
|-------------|------------------------------------------------------|-------------------|---------------------|
| JNA         | Jump if <b>N</b> ot <b>A</b> bove                    | CF = 1 ou ZF = 1  | JA                  |
| JNAE        | Jump if <b>N</b> ot <b>A</b> bove or <b>E</b> qual   | CF = 1            | JAE                 |
| JNB         | Jump if <b>N</b> ot <b>B</b> elow                    | CF = 0            | JB                  |
| JNBE        | Jump if <b>N</b> ot <b>B</b> elow or <b>E</b> qual   | CF = 0 et ZF = 0  | JBE                 |
| JNC         | Jump if <b>N</b> o <b>C</b> arry                     | CF = 0            | JC                  |
| JNE         | Jump if <b>N</b> ot <b>E</b> qual                    | ZF = 0            | JE                  |
| JNG         | Jump if <b>N</b> ot <b>G</b> reater                  | ZF = 1 ou SF ≠ OF | JG                  |
| JNGE        | Jump if <b>N</b> ot <b>G</b> reater or <b>E</b> qual | SF ≠ OF           | JGE                 |
| JNL         | Jump if <b>N</b> ot <b>L</b> ess                     | SF = OF           | JL                  |
| JNLE        | Jump if <b>N</b> ot <b>L</b> ess or <b>E</b> qual    | ZF = 0 et SF = OF | JLE                 |





## Les sauts conditionnels

### Les mnémoniques de sauts conditionnels

| Instruction | Description                          | Condition | Instruction opposée |
|-------------|--------------------------------------|-----------|---------------------|
| JNO         | Jump if <b>N</b> ot <b>O</b> verflow | OF = 0    | JO                  |
| JNP         | Jump if <b>N</b> o <b>P</b> arity    | PF = 0    | JP                  |
| JNS         | Jump if <b>N</b> o <b>S</b> ign      | SF = 0    | JS                  |
| JNZ         | Jump if <b>N</b> ot <b>Z</b> ero     | ZF = 0    | JNZ                 |
| JO          | Jump if <b>O</b> verflow             | OF = 1    | JNO                 |
| JP          | Jump if <b>P</b> arity               | PF = 1    | JNP                 |
| JPE         | Jump if <b>P</b> arity <b>E</b> ven  | PF = 1    | JPO                 |
| JPO         | Jump if <b>P</b> arity <b>O</b> dd   | PF = 0    | JPE                 |
| JS          | Jump if <b>S</b> ign                 | SF = 1    | JNS                 |
| JZ          | Jump if <b>Z</b> ero                 | ZF = 1    | JNZ                 |





# Les sauts conditionnels

## La limitation dans l'amplitude des sauts

- L'amplitude des sauts conditionnels est limitée à 127 octets vers l'avant et 128 octets vers l'arrière.
- Pour contourner ce problème (nous affranchir de cette limite), nous pouvons coupler ce saut conditionnel avec un saut inconditionnel.

```
...
 CMP AX, BX
 JZ grandSaut
 JMP loin1
grandSaut: JMP loin2
...
loin1: ...
loin2: ...
```





# Les sauts conditionnels

## Exercice 1

- Traduire le code C ci-contre en assembleur avec :
  - CMP ;
  - JNZ ;
  - JMP.

```
if (ax == 4h)
{
 bx = 10h;
}
else {
 bx = 20h;
 cx = 30h;
}
```





# Les sauts conditionnels

## Exercice 2

- Traduire le code C ci-contre en assembleur avec :
  - CMP ;
  - JA ;
  - INC ;
  - JMP.

```
bx=0;

for (cx=0; cx<10; cx++)
{
 bx += cx;
}
```





# Les sauts conditionnels

## Exercice 3

- Traduire le code C ci-contre en assembleur avec :
  - CMP ;
  - JLE ;
  - DEC ;
  - JMP.

```
bx=5;

while (bx>0)
{
 bx--;
}
```





# Les sauts conditionnels

## Exercice 4

- Traduire le code C ci-contre en assembleur avec :

- DEC ;
- CMP ;
- JG.

```
bx=10;

do
{
 bx--;
}
while (bx>0);
```





# Les sauts conditionnels

## Exercice 5

- Traduire le code C ci-contre en assembleur avec :

- CMP ;
- JNZ ;
- JMP.

```
switch (bx)
{
 case 1 : ax=1;
 break;
 case 2 : ax=5;
 break;
 case 3 : ax=9;
 break;
 default : ax=0;
};
```







# Les boucles

## Le registre CX comme compteur

- Nous avons vu une première manière d'implanter des boucles en utilisant les mnémoniques de sauts conditionnels (JNZ, JMP ...) : cela se rapproche du « while » et du « do ... while ».
- La seconde méthode (qui se rapproche plutôt du « for ») consiste à utiliser le registre CX comme un compteur par l'intermédiaire d'une des mnémoniques suivantes :
  - LOOP ;
  - LOOPE ;
  - LOOPNE ;
  - LOOPNZ ;
  - LOOPZ ;
  - JCXZ.





# Les boucles

## Les mnémoniques CLD, CLI, STD et STI

| Instruction | Description                                                     |
|-------------|-----------------------------------------------------------------|
| LOOP        | Décrémente CX et saute vers le label si $CX \neq 0$             |
| LOOPE       | Décrémente CX et saute vers le label si $CX \neq 0$ et $ZF = 1$ |
| LOOPNE      | Décrémente CX et saute vers le label si $CX \neq 0$ et $ZF = 0$ |
| LOOPNZ      | Décrémente CX et saute vers le label si $CX \neq 0$ et $ZF = 0$ |
| LOOPZ       | Décrémente CX et saute vers le label si $CX \neq 0$ et $ZF = 1$ |
| JCXZ        | Saute vers le label si $CX = 0$                                 |

- Ces instructions, à l'exception de la dernière, décrémente la valeur contenue dans le registre CX.





# Les boucles

## Les mnémoniques CLD, CLI, STD et STI

- La construction d'une boucle peut s'effectuer simplement en respectant quelques contraintes :
  - charger la valeur de la boucle dans le registre CX ;
  - faire précéder la première instruction de la boucle par une étiquette ;
  - terminer la boucle par une instruction de type LOOP qui pointe sur l'étiquette.





# Les boucles

## Exemple

- Nous obtenons alors une suite d'instructions qui peut ressembler à celle ci-dessous :

```
 MOV CX, 5
boucle: INC AX
 ... etc
 LOOP boucle ; Si CX != 0, on reboucle

 ...etc
```





# Les boucles

## Les boucles imbriquées

- Il est possible de créer plusieurs boucles imbriquées mais il faut constamment sauvegarder l'état du registre CX pour passer d'une boucle à l'autre.
- Nous verrons un exemple d'une telle manipulation lorsque nous aborderons les instructions **PUSH** et **POP** qui permettent de manipuler la pile (le segment SS).





# Pause-réflexion sur cette 4<sup>ème</sup> partie

**Avez-vous des questions ?**

